Algorithms 2
Appunti

Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Matroids and Matroid Intersections** Page

1.1 Matroids
Greedy Algorithm for Max Weight Spanning Trees — • Matroids and Greedy — • Examples of matroids —

1.2 Matroid Intersection
Examples —

1.3 Algorithm for Maximum Cardinality Bipartite Matching
Duality — • Algorithm — • Proof of König's Theorem —

Chapter 2 **Linear Programming** Page

2.1 Introduction
Extreme Points —

2.2 Examples
Maximum Weight Bipartite Perfect Matching — • Bipartite Vertex Cover —

2.3 Duality
Intuition — • General case — • Duality Theorems — • Example: Maximum cardinality matching and Vertex Cover on Bipartite Graphs — • Complementarity Slackness —

2.4 Hungarian Method
Duality and Complementarity Slackness of Min-Cost Perfect Matching — • The Hungarian Algorithm —

Chapter 3 **Approximation Algorithms** Page

3.1 When Relaxing Integrality is Not for Free: Approximation Algorithms

3.2 Using Linear Programming to Design Approximation Algorithms
Vertex Cover — • Integrality Gap —

3.3 Set Cover via Randomized Rounding
A better approximation for Set Cover —

3.4 Integrality gap of the set cover LP

Chapter 4 **The Multiplicative Weights Algorithm** Page

4.1 Naive Implementations
Warmup: Prediction with Expert Advice — • Fewer Mistakes with Weighted Majority —

4.2 Randomized Strategies

4.3 The Hedge strategy

4.4 Hedging for LPs
Covering LPs — • General idea — • Implementation of the oracle — • Hedge algorithm for covering LPs —

Chapter 1

Matroids and Matroid Intersections

1.1 Matroids

1.1.1 Greedy Algorithm for Max Weight Spanning Trees

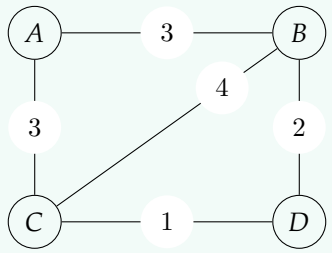
Let's start by getting a better understand perhaps the easiest of algorithms: always select the best available option in a greedy way. We first describe the algorithm when considering the spanning tree problem or rather the maximum weight forest problem. First recall the definition of the maximum weight spanning tree problem:[1]

Definition 1.1.1: Max Weight Spanning Trees

Given a connected undirected graph $G = (V, E)$ with edge weights $w : E \to \mathbb{R}$, find a spanning tree $T \subseteq E$ of maximum total weight $w(T) := \sum_{e \in T} w(e)$.

Example 1.1.1

The following undirected edge-weighted graph has two maximum spanning trees: $\{\{A, B\}, \{B, C\}, \{B, D\}\}$ and $\{\{A, C\}, \{B, C\}, \{B, D\}\}$.



We are now going to describe the most basic greedy algorithm there is: repeatedly add the edge of largest weight that does not create a cycle. When considering the spanning tree problem this is known as Kruskal's algorithm (named after its inventor).

[1] It is perhaps more common to consider the minimum spanning tree problem (which is equivalent by replacing each weight $w(e)$ by $-w(e)$). Here we consider the maximum weight spanning tree problem for the purpose of generalizing it to matroids.

Greedy(G, w)**:**
Input: A connected undirected graph $G = (V, E)$ and weights $(w_e)_{e \in E}$.
Output: A maximum weight spanning tree S.

1. Sort and relabel the edges so that $w_{e_1} \geq w_{e_2} \geq \cdots \geq w_{e_{|E|}}$.
2. $S \leftarrow \emptyset$.
3. **for** $i = 1$ **to** $|E|$:
4. **if** $S + e_i$ is acyclic **then** $S \leftarrow S + e_i$.
5. **return** S.

Runtime analysis. Steps 3–4 can be implemented in almost linear time using the Union-Find (also called Disjoint-Set) data structure. The running time is thus dominated by the sorting of Step 1. Using e.g. Merge-Sort this step runs in time $\Theta(|E| \log |E|)$. The total running time is thus $\Theta(|E| \log |E|)$.

Why does it work? The correctness of the algorithm follows from the following lemma.

Lenma 1.1.1

Greedy returns a maximum-weight spanning tree.

: Suppose not. Let $S = \{s_1, s_2, \ldots, s_{n-1}\}$ be the set (spanning tree) returned by the algorithm and suppose that a solution T has a higher weight, where $T = \{t_1, t_2, \ldots, t_{n-1}\}$ (indexed in decreasing weight). Let p be the first index such that $w(t_p) > w(s_p)$. Let $A = \{t_1, \ldots, t_p\}$ and $B = \{s_1, \ldots, s_{p-1}\}$. Now we have the following key property of acyclic graphs.

Key property: As $|A| > |B|$, there exists $e \in A \setminus B$ such that $B + e$ is acyclic.

The key property follows from that an acyclic graph with k edges has $n - k$ components (provable by induction on k). Thus the graph (V, A) has fewer components than (V, B) and so at least one edge $e \in A$ must connect two different components of (V, B). It follows that $e \notin B$ and that $B + e$ is acyclic.

Having proved the key property, we conclude as follows. Since $w(e) \geq w(t_p) > w(s_p)$, e should have been selected when it was considered.
To be more precise and detailed, when e was considered, the greedy algorithm checked whether e could be added to the current set at the time, say B'. But since $B' \subseteq B$, adding e to B' would have resulted in an acyclic graph (a subset of acyclic edges is also acyclic) since its addition to B results in an acyclic graph.
This gives a contradiction and completes the proof.

1.1.2 Matroids and Greedy

Note that in the correctness of the Greedy algorithm for max-weight spanning trees (Lemma 1.1.1) we used two properties of acyclic graphs: (i) a subset of acyclic edges is acyclic and (ii) for two acyclic edge sets A, B on the same vertex-set with $|A| > |B|$ there is $e \in A \setminus B$ such that $B + e$ is acyclic.
Matroids are defined to satisfy these two properties and we will see that they define the set of problems for which the basic greedy algorithm works. It also generalizes the notion of linear independence in matrices. There are many equivalent definitions. We use the one that focus on its independent sets.

Definition 1.1.2: Matroid

A matroid $M = (E, \mathcal{I})$ is defined on a finite ground set E and a family $\mathcal{I}$ of subsets of E that are called independent sets satisfying two axioms:

(I1) if $X \subseteq Y$ and $Y \in \mathcal{I}$ then $X \in \mathcal{I}$.

(I2) if $X \in \mathcal{I}$ and $Y \in \mathcal{I}$ and $|Y| > |X|$ then $\exists e \in Y \setminus X : X \cup \{e\} \in \mathcal{I}$.

Remarks:

- Letting E be the edges of a graph and $\mathcal{I} = \{F \subseteq E : F \text{ is acyclic}\}$ defines a so-called *graphic matroid*. Note that it satisfies (I1) since the subset of an acyclic set of edges is acyclic and it satisfies (I2) due to the key property used in the proof of Lemma 1.1.1. We see more examples of matroids in Section 1.1.3.
- The family $\mathcal{I}$ may be exponential in the size of the ground set E (as is the case for example for graphic matroids of complete graphs). One therefore often assumes that $\mathcal{I}$ is given implicitly by having a *membership oracle*: an algorithm that given $I \subseteq E$ efficiently answers whether $I \in \mathcal{I}$.

Definition 1.1.3: Matroid Base

A *base* of a matroid is an independent set of maximum cardinality.

Claim 1.1.1 Maximality of bases

All maximal independent sets are bases and have the same cardinality.

: Implied by the second axiom.

With the more abstract notation of matroids, the basic greedy algorithm becomes

Greedy(M, w)**:**
Input: A matroid $M = (E, \mathcal{I})$ and weights $(w_e)_{e \in E}$.
Output: A maximum weight base S.

1. Sort and relabel the elements so that $w_1 \geq w_2 \geq \cdots \geq w_{|E|}$.
2. $S \leftarrow \emptyset$.
3. **for** $i = 1$ **to** $|E|$:
4. **if** $S + i \in \mathcal{I}$ **then** $S \leftarrow S + i$.
5. **return** S.

So Greedy returns the maximum cardinality set in $\mathcal{I}$ that has maximum weight.
The concept of matroids was defined so as to enable the correctness analysis of the basic greedy algorithm. Perhaps more surprisingly, it is if and only if.

Theorem 1.1.1 Rado'57/Gale'68/Edmonds'71

For any ground set $E = \{1, 2, \ldots, n\}$, and a family of subsets $\mathcal{I}$, Greedy finds a maximum weight base for any set of weights $w : E \to \mathbb{R}$ if and only if $M = (E, \mathcal{I})$ is a matroid.

The if direction ($\Leftarrow$ part) follows from Lemma 1.1.1. For the only if direction we have the following claim.

Claim 1.1.2 $\Rightarrow$ part

Suppose $(E, \mathcal{I})$ is not a matroid. There exists an assignment of weights $w : E \to \mathbb{R}$ so that Greedy does not return a maximum weight base.

: If $(E, \mathcal{I})$ is not a matroid, then it violates at least one of the two axioms.

- First suppose $\mathcal{I}$ is not a downward-closed family of sets, i.e., it violates (I1). Therefore, there exist two sets $S \subset T$ such that $S \notin \mathcal{I}$ and $T \in \mathcal{I}$. Consider the following weights:

$$w_i = \begin{cases} 2 & i \in S, \\ 1 & i \in T \setminus S, \\ 0 & \text{otherwise.} \end{cases}$$

  By the weight assignment, the algorithm first considers the elements of S, then the elements of T, and then the rest of the elements. Suppose the algorithm selects a subset S_1 of S. Since $S \notin \mathcal{I}$, S_1 is a strict subset. Out of the remaining elements the algorithm can select at most $T \setminus S$. So the weight of the independent set that the algorithm returns is at most $2|S_1| + |T \setminus S|$ which is less than $w(T)$ so it is not a maximum weight independent set.

- Second suppose that the extension axiom is violated (but downwardness is satisfied). In particular, let $S, T \in \mathcal{I}$ be two independent sets such that $|S| < |T|$, and for all $i \in T \setminus S$, $S + i \notin \mathcal{I}$. Now use the following weights

$$w_i = \begin{cases} 1 + \frac{1}{2|S|} & i \in S, \\ 1 & i \in T \setminus S, \\ 0 & \text{otherwise.} \end{cases}$$

  Because of downwardness the algorithm would select all elements in S and return an independent set of value $|S| + 1/2$ where as the optimal set would have value at least $|T| > |S| + 1/2$.



1.1.3 Examples of matroids

We already saw the graphic matroids. Other basic matroids are as follows:

k-Uniform matroid

A matroid $M = (E, \mathcal{I})$ is k-Uniform if $\mathcal{I}$ satisfies:

$$\mathcal{I} = \{X \subseteq E : |X| \leq k\}.$$

Partition matroid

A matroid $M = (E, \mathcal{I})$ is a partition matroid if E is partitioned into disjoint sets $E_1, E_2, \ldots, E_\ell$ and

$$\mathcal{I} = \{X \subseteq E : |E_i \cap X| \leq k_i \text{ for } i = 1, 2, \ldots, \ell\}.$$

Linear matroid

A matroid $M = (E, \mathcal{I})$ is a linear matroid when it is defined from a matrix A. Let E be the index set of the columns and for $X \subseteq E$ let A_X be the matrix consisting of the columns indexed by X. Define $\mathcal{I}$ by

$$\mathcal{I} = \{X \subseteq E : \text{rank}(A_X) = |X|\}.$$

So $\mathcal{I}$ is the set of all *linearly independent* subsets of the column vectors.

Truncated matroid

A truncated matroid $M_k = (E, \mathcal{I}_k)$ is defined from a matroid $M = (E, \mathcal{I})$ such that

$$\mathcal{I}_k = \{X \in \mathcal{I} : |X| \leq k\}.$$

It is quite easy to verify that the axioms still hold for M_k, as $X \in \mathcal{I}_k$ implies $X \in \mathcal{I}$ for all $X \subseteq E$.

(I1) holds because $B \in \mathcal{I}_k$ means that $|B| \leq k$ and $A \subseteq B$ thus means $|A| \leq k$ as well. We know that M is a matroid so (I1) holds for M, which implies $A \in \mathcal{I}$. We conclude that $A \in \mathcal{I}_k$.
The same reasoning can verify (I2): if $A, B \in \mathcal{I}_k$ and $|B| > |A|$, then $|B| \leq k$ and $|A| \leq k-1$. We know that (I2) holds for M, so the inclusion of $\mathcal{I}_k$ in $\mathcal{I}$ tells us that $\exists e \in B \setminus A$ such that $A + e \in \mathcal{I}$. The fact that $|A + e| \leq k - 1 + 1 = k$ allows us to conclude.

Note:

This tells us that the intermediate stepps of GREEDY are still bases of the truncation of the original matroid.

Half a Bipartite Matching

Given a bibartite graph $G = (A \cup B, E)$, let $(A, \mathcal{I})$ be the matroid s.t.

$$\mathcal{I} = \{A' \subseteq A : G \text{ has a matching s.t. every vertex in } A \text{ is matched}\}$$

Laminar Matroid

Consider a family $\mathcal{F}$ of subsets of the ground set E that satisfies $\forall X, Y \in \mathcal{F}$:

- $X \cap Y = \emptyset$
- $X \subseteq Y$
- $Y \subseteq X$

For any positive integers $\{k_X\}_{X \in \mathcal{F}}$ (one for each set in $\mathcal{F}$) we have that $\mathcal{M} = (E, \mathcal{I})$ is a matroid, where

$$\mathcal{I} = \{S \subseteq E : |S \cap X| \leqslant k_X \quad \forall X \in \mathcal{F}\}.$$

1.2 Matroid Intersection

Although the concept of matroids forms a rich set of problems that can be solved by the greedy algorithm, there are many problems that have efficient algorithms but are not matroids. Consider for example the bipartite matching problem:

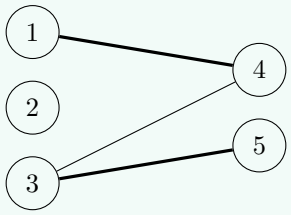
Definition 1.2.1: Max Size Matching

Given a bipartite graph $G = (V, E)$, find a matching of maximum size.

Recall that a matching $M \subseteq E$ is a subset of edges so that every vertex is incident to at most one edge of M, i.e., $|\{e \in M : v \in e\}| \leq 1$ for all $v \in V$.

Example 1.2.1

The edges $(1, 4)$ and $(3, 5)$ form a matching (of maximum cardinality).



Let's see if Greedy works for maximum cardinality (or max weight) bipartite matching. To do this, we can verify if $(E, \mathcal{I})$ is a matroid where

$$\mathcal{I} = \{M \subseteq E : M \text{ is a matching}\}.$$

It is easy to see that $(E, \mathcal{I})$ satisfies the downward closed property (I1) of matroids since a subset of a matching is still a matching. However, the second axiom (I2) is not satisfied which can be seen by taking the two matchings $M_1 = \{(1, 4), (3, 5)\}$ and $M_2 = \{(3, 4)\}$: although $|M_1| > |M_2|$ there is no edge in M_1 that we can add to M_2 while maintaining that it is a matching.

This is kind of sad: matroids and simple Greedy have their limits. On the other hand, we know from our undergraduate algorithm course that there are efficient algorithms for the bipartite matching problem. That there are efficient algorithms for the bipartite matching problem is in fact part of a more general phenomena: it can be described as the intersection of two matroids and any such problem has an efficient algorithm!

Definition 1.2.2: Matroid Intersection

Given two matroids $\mathcal{M}_1 = (E, \mathcal{I}_1)$ and $\mathcal{M}_2 = (E, \mathcal{I}_2)$ on the same ground set E, their intersection is

$$\mathcal{M}_1 \cap \mathcal{M}_2 = (E, \mathcal{I}_1 \cap \mathcal{I}_2).$$

Note that the intersection of two matroids satisfy (I1) but not typically (I2). The following adds a lot of power to the concepts of matroids:

Theorem 1.2.1 Edmonds, Lawler, 70's

There is an efficient algorithm for finding a max-weight independent set in the intersection of two matroids.[a]

[a] It is efficient meaning that it is a polynomial-time algorithm if we can, in polynomial time, answer independence queries of the type "$I \in \mathcal{I}_1$?" and "$I \in \mathcal{I}_2$?" for the two matroids (which is the case for all examples of matroids seen in class).

The algorithm that gives the above result is quite technical and notation heavy. Instead of describing it in its full generality, we first see how we can apply the above result to obtain several interesting algorithmic applications. We then develop an algorithm for the special case of the bipartite matching problem that share several similarities with the general algorithm.

1.2.1 Examples

From http://www-math.mit.edu/~goemans/18433S09/matroid-intersect-notes.pdf

Bipartite Matching

We already mentioned that the bipartite matching problem is an example of matroid intersection. To see this, consider a bipartite graph $G = (V, E)$ with bipartition (A, B).
Let $\mathcal{M}_A$ be a partition matroid with ground set E where the partition E is given by $E = \bigcup\{\delta(v) : v \in A\}$ where $\delta(v)$ denotes the edges incident to v. Notice that this is a partition since all edges have precisely one endpoint in A. We also define $k_v = 1$ for every $v \in A$. Thus, the family of independent sets of $\mathcal{M}_A$ is given by

$$\mathcal{I}_A = \{F \subseteq E : |F \cap \delta(v)| \leq 1 \text{ for all } v \in A\}.$$

In other words, a set of edges is independent for $\mathcal{M}_A$ if it has at most one edge incident to every vertex of A (and any number of edges incident to every vertex of B). We can similarly define $\mathcal{M}_B = (E, \mathcal{I}_B)$ by

$$\mathcal{I}_B = \{F \subseteq E : |F \cap \delta(v)| \leq 1 \text{ for all } v \in B\}.$$

Now observe that any $F \in \mathcal{I}_A \cap \mathcal{I}_B$ corresponds to a matching in G, and vice versa. And the largest common independent set $\mathcal{I}_A$ and $\mathcal{I}_B$ corresponds to a maximum matching in G.

Colorful Spanning Trees

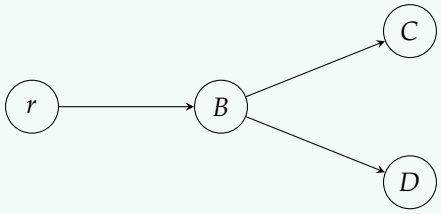
Suppose we have an undirected graph $G = (V, E)$ and every edge has a color. This is represented by a partition of E into $E_1 \cup E_2 \cup \cdots \cup E_k$ where each E_i represents a set of edges of the same color i. The problem of deciding whether this graph has a spanning tree in which all edges have a different color can be tackled through matroid intersection. Such a spanning tree is called *colorful*. Colorful spanning trees are bases of the graphic matroid $\mathcal{M}_1 = (E, \mathcal{I}_1)$ of G which are also independent in the partition matroid $\mathcal{M}_2 = (E, \mathcal{I}_2)$ defined by $\mathcal{I}_2 = \{F \subseteq E : |F \cap E_i| \leq 1 \text{ for all } i\}$.

Arborescences

Given a directed graph $D = (V, A)$ and a special root vertex $r \in V$, an *r-arborescence* (or just arborescence) is a spanning tree (when viewed as an undirected graph) directed away from r. Thus, in a r-arborescence, every vertex is reachable from the root r. As an r-arborescence has no arc incoming to the root, we assume that D has no such arc.

Example 1.2.2

The following directed graph has e.g. the r-arborescence $\{(r, B), (B, C), (B, D)\}$.



r-arborescences can be viewed as sets simultaneously independent in two matroids. Let G denote the undirected counterpart of D obtained by disregarding the directions of the arcs. Note that if we have both arcs $a_1 = (u, v)$ and $a_2 = (v, u)$ in D then we get two undirected edges also labeled a_1 and a_2 between u and v in G. Define $\mathcal{M}_1 = (A, \mathcal{I}_1)$ the graphic matroid corresponding to G, and $\mathcal{M}_2 = (A, \mathcal{I}_2)$ the partition matroid in which independent sets are those with at most one arc incoming to every vertex $v \neq r$. In other words, we let

$$\mathcal{I}_2 = \{F : |F \cap \delta^-(v)| \leq 1 \text{ for all } v \in V \setminus \{r\}\},$$

where $\delta^-(v)$ denotes the set $\{(u, v) \in A\}$ of arcs incoming to v. Thus, any r-arborescence is independent in both matroids $\mathcal{M}_1$ and $\mathcal{M}_2$. Conversely, any set T independent in both $\mathcal{M}_1$ and $\mathcal{M}_2$ and of cardinality $|V| - 1$ (so that it is a base in both matroids) is an r-arborescence. Indeed, such a T being a spanning tree in G has a unique path between r and any vertex v; this path must be directed from the root r since otherwise we would have either an arc incoming to r or two arcs incoming to the same vertex.

1.3 Algorithm for Maximum Cardinality Bipartite Matching

1.3.1 Duality

Before giving an algorithm, perhaps an easier question is: how would you give a short proof that a given matching is optimal?
For this purpose, one would like to find upper bounds on the size of the largest matching and hope that the smallest of these upper bounds be equal to the size of the largest matching. This is a duality concept that will be very important in this subject. In this case, the dual problem will be a famous combinatorial optimization problem: *vertex cover*.

Definition 1.3.1: Vertex cover

A vertex cover is a set C of vertices so that all edges e of E are incident to at least one vertex in C. In other words, there is no edge completely contained in $V \setminus C$.

We clearly have the following:

$$|M| \leq |C| \quad \text{for any matching } M \text{ and vertex cover } C.$$

This follows from the fact that, given any matching M, a vertex cover C must contain at least one of the end points of each edge in M.
This is *weak duality*: The maximum size of a matching is at most the minimum size of a vertex cover. We shall in fact prove *strong duality* (that equality holds) for bipartite graphs:

Theorem 1.3.1 König 1931

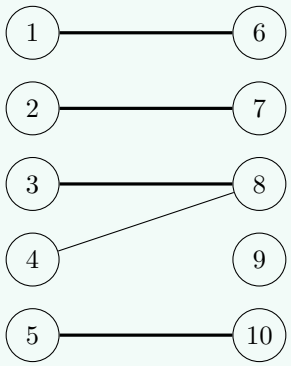
Let $M^\star$ be a maximum cardinality matching and $C^\star$ be a minimum vertex cover of a bipartite graph. Then

$$|M^\star| = |C^\star|.$$

The proof of this theorem will be algorithmic. It will give an efficient algorithm for both finding a maximum size matching and a minimum size vertex cover of a bipartite graph. Note that whenever one has min max statement as above, the problem lies in $\mathsf{NP} \cap \mathsf{coNP}$ which may indicate that it has an efficient algorithm.

Example 1.3.1

The edges $(1,6),(2,7),(3,8)$ and $(5,10)$ form a matching (of maximum cardinality). Vertices $1, 2, 5$, and 8 form a vertex cover (of minimum cardinality).



1.3.2 Algorithm

Recall that a path is a collection of edges $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$ where the v_i's are distinct vertices. We can simply represent a path as $v_0 - v_1 - v_2 - \ldots - v_k$.

Definition 1.3.2: Alternating path

An *alternating path* with respect to M is a path that alternates between edges in M and edges in $E \setminus M$.

Definition 1.3.3: Augmenting path

An *augmenting path* with respect to M is an alternating path in which the first and last vertices are unmatched.

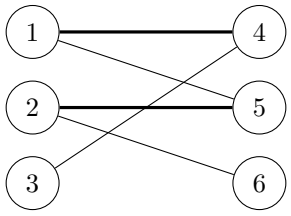


Figure 1.1: The edges $(3,4),(4,1),(1,5),(5,2),(2,6)$ form an augmenting path

The definition of an augmenting path motivates the following algorithm:

AugmentingPathAlgorithm(G)**:**
Input: A bipartite graph $G = (V, E)$.
Output: A matching M of maximum cardinality.

1. Initialize $M = \emptyset$.
2. **while** exists an augmenting path P
3. update $M = M \Delta P \equiv (M \setminus P) \cup (P \setminus M)$.
4. **return** M.

Example 1.3.2 (Exercise)

Devise an efficient algorithm for finding an augmenting path P (if one exists). What is the total running time of the AugmentingPathAlgorithm?

Possible algo:

1. Create a directed graph where the edges in M go from B to A and the others are reversed. Connect a new vertex s to all vertexes in A and all vertexes in B to a new vertex t.
2. Run a BFS from s to t on this new graph.
3. If t is reachable, the path (mins $s \to A$ and $B \to t$) is augmenting. Else, no augmenting path exists.

We now prove that AugmentingPathAlgorithm indeed finds a maximum matching.

Theorem 1.3.2

A matching M is maximum if and only if there are no augmenting paths with respect to M.

: ($\Rightarrow$) Let P be some augmenting path with respect to M. Then $M' = M \Delta P$ is matching of greater cardinality than M. This contradicts the optimality of M.
($\Leftarrow$) If M is not maximum, let M^* be a maximum matching so that $|M^*| > |M|$. Let $Q = M \Delta M^*$. Then

- Q has more edges from M^* than from M (since $|M^*| > |M|$ implies that $|M^* \setminus M| > |M \setminus M^*|$).
- Each vertex is incident to at most one edge in $M \cap Q$ and one edge in $M^* \cap Q$.
- Thus Q is composed of cycles and paths that alternate between edges from M and M^*.
- Therefore there must be some path with more edges from M^* in it than from M. This path is an augmenting path with respect to M.

Hence, there must exist an augmenting path P with respect to M, which is a contradiction. ✆

1.3.3 Proof of König's Theorem

Consider a maximum matching M^* of $G = (A \cup B, E)$. Let L the vertices reachable from unmatched vertices in A by alternating paths with respect to M^*. (So e.g. in the figure of Example 1.3.1 $L = \{4, 8, 3\}$.) The following proves König's theorem.

Lenma 1.3.1

We have that $C^* = (A \setminus L) \cup (B \cap L)$ is a vertex cover. Moreover, $|C^*| = |M^*|$.

: We first show that C^* is a vertex cover. Suppose toward contradiction that it is not. Then there is an edge $e = (a, b) \in E$ with $a \in A \cap L$ and $b \in B \setminus L$. The edge cannot belong to the matching. If it did then b should be in L because otherwise a would not be in L. Hence $e \in E \setminus M$. This however, implies that there is an alternating path from an unmatched vertex to b (namely go to a and take the edge (a, b)) contradicting the fact that $b \notin L$. To show the second part of the proof, we show that $|C^*| \leq |M^*|$, since the reverse inequality is true for any matching and any vertex cover. The proof follows from the following observations:

1. No vertex in $A \setminus L$ is unmatched by the definition of L;
2. No vertex in $B \cap L$ is unmatched since this would imply the existence of an augmenting path (which contradicts that M^* is optimal).
3. There is no edge of the matching between a vertex $a \in A \setminus L$ and a vertex $b \in B \cap L$. Otherwise a would be in L.

These remarks imply that every vertex in C^* is matched and moreover the corresponding edges of the matching are distinct. Hence, $|C^*| \leq |M^*|$. ✆

Chapter 2

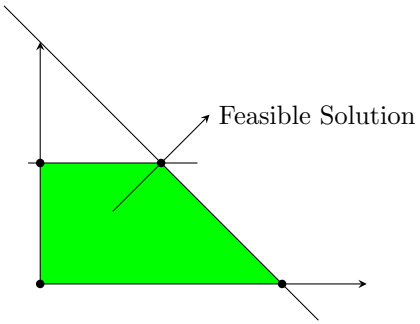
Linear Programming

2.1 Introduction

A linear programing problem is the problem of finding values for variables that optimize a given linear objective function, subject to linear constraints.
Let us state an example of a linear program:

$$\begin{array}{ll} \text{Maximize} & x + y \\ \text{Subject to} & x + y \leq 2 \\ & y \leq 1 \\ & x, y \geq 0 \end{array}$$

The following figure shows the feasible area.



Definition 2.1.1: Linear Program

A *linear program (LP)* is the problem of finding values for n variables $x_1, x_2, \ldots, x_n \in \mathbb{R}$ that minimize (or equivalently, maximize) a given linear objective function, subject to m linear constraints

$$\begin{array}{lll} \textbf{minimize:} & \sum_{i=1}^{n} c_i x_i & \\ \textbf{Subject to:} & \sum_{i} e_{i,j} x_i = b_j & \text{for } j = 1, \ldots, m_1 \\ & \sum_{i} d_{i,k} x_i \geq g_k & \text{for } k = 1, \ldots, m_2 \\ & \sum_{i} f_{i,p} x_i \leq l_p & \text{for } p = 1, \ldots, m_3 \end{array}$$

where $m_1 + m_2 + m_3 = m$.

Motivation

Linear Programming is a very powerful tool – it can be used in many applications, both industrial and theoretical such as:

- obtaining an optimal production plan to maximize a profit of a factory
- modeling network flow problems (what is maximum possible flow that doesn't exceed capacity of each connection)
- theory – many theoretical problems can be introduced as Integer Programs (LP in which $x_i \in \mathbb{Z}$) that can be relaxed to LP obtaining a good approximation of optimal result – we'll see such applications later in the course.

Solving Methods

Simplex Method was published by George Dantzig in 1947: it is the first complete algorithm to solve linear programming problems.
The principle is the following: we start from an extreme point and then we look at its neighbors. If one of these is better we move to it and continue in the same way, else we stop. Once we stop, we can be sure that we have an optimal solution, since we're in a convex polytope. Even if it's usually extremely fast, we know some bad examples where this method visits an exponential number of extreme points before to reach a solution, so this method does not always run in polynomial time.

Ellipsoid Method was studied by Leonid Khachiyan in the seventies.
This method is guaranteed to run in poly time (exactly poly$(n, m, \log(u))$ where u is the largest constant) but it is slow in practice.
We do a binary search for the optimal value of the objective function, so we can add the objective function as a constraint, and just need to decide if there exists a feasible point. We start by taking an ellipsoid surrounding the feasible area. We then check the center of the ellipsoid, if it's inside the area then we have our solution, else we identify a violated constraint, and cut our ellipsoid in two parts using this constraint, and construct a new ellipsoid around the part of the old ellipsoid in which the constraint is satisfied. We repeat this process until we find a feasible point or can be sure that the feasible area is empty.

Interior point Method developed by Narendra Karmarkar in 1984. In this method we move in the feasible region to find an OPT solution.

2.1.1 Extreme Points

Let us first define an extreme point:

Definition 2.1.2: Extreme Point

A feasible solution is an *extreme point* if it cannot be written as a convex combination of other feasible solutions.

Just to recall:

Definition 2.1.3: Convex Combination

A *convex combination* of points $x_1, x_2, \ldots, x_n$ is a point of the form $\sum_{i=1}^{n} \lambda_i x_i$ where the real numbers λ_i satisfy $\sum_{i=1}^{n} \lambda_i = 1$ and $\lambda_i \in [0, 1]$ for all i.

Now, we state a theorem about extreme points. Extreme points are important because sometimes they have useful structural properties, which we can exploit to design algorithms. We see two such examples in this lecture.

Theorem 2.1.1

If the feasible region is bounded, there always exists an optimum which is an extreme point.

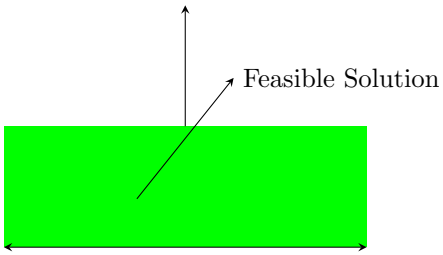
: As the feasible region is bounded, there is an optimal solution $x^\star$ and any feasible point can be written as a convex combination of the extreme points. In particular, we have $x^\star = \sum_i \lambda_i x^{(i)}$ where $x^{(i)}$'s are feasible extreme points and λ_i's are non-negative real numbers satisfying $\sum_i \lambda_i = 1$. Now let c be the vector defining the objective that we wish to maximize (the proof is the same if we minimize). Then we have

$$c^T x^\star = c^T \left(\sum_i \lambda_i x^{(i)} \right) = \sum_i \lambda_i c^T x^{(i)},$$

which implies that there is an extreme point $x^{(i)}$ such that $c^T x^{(i)} \geq c^T x^\star$, i.e., it is an optimal solution. ☺

If the feasible region is not bounded, then we might not have any extreme points. For example, the following LP does not contain any extreme points.

$$\begin{array}{ll} \text{Maximize} & y \\ \text{Subject to} & y \leq 1 \\ & y \geq 0 \end{array}$$



2.2 Examples

2.2.1 Maximum Weight Bipartite Perfect Matching

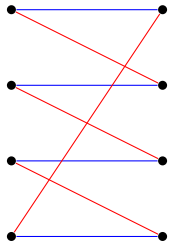
In this section we are going to concentrate on maximum weight/profit bipartite perfect matching. Given a bipartite graph $G = (V, E)$ with V partitioned into A and B and edge-weights $w : E \to \mathbb{R}$, this is the problem where we wish to find a perfect matching M that maximizes $w(M) = \sum_{e \in M} w(e)$. The LP for the maximum weight bipartite perfect matching problem can be formulated as

$$\begin{array}{lll} \text{Maximize} & \displaystyle\sum_{e \in E} x_e w_e & \\ \text{Subject to} & \displaystyle\sum_{e=(a,b) \in E} x_e = 1 & \forall a \in A \\ & \displaystyle\sum_{e=(a,b) \in E} x_e = 1 & \forall b \in B \\ & x_e \geq 0 & \forall e \in E \end{array}$$

In the linear program, we have a variable x_e for each edge e with the intended meaning that it should take value 1 if that edge is picked by the matching. The constraints say that each vertex should have exactly one incident edge in the matching.
Note that in a feasible solution the variables x_e may take values strictly between 0 and 1, which does not really correspond to a matching. However, we have the following key structural result:

> **Claim 2.2.1**
> For bipartite graphs, any extreme point solution to the LP is integral.

: Let x^* be an extreme point for the graph $G = (V_1, V_2, E)$ and let $E_f = \{e \in E : 0 < x^*_e < 1\}$. Suppose towards contradiction that $E_f \neq \emptyset$. Note that E_f must then contain a cycle: indeed any vertex incident to an edge in E_f is incident to at least two edges in E_f. All these edges are fractional and we



want to define y and z so that they are feasible solutions and $x^* = \frac{1}{2}(y+z)$ which will contradict the fact that x^* is an extreme point. Let $e_1, e_2, \ldots, e_{2k}$ be the edges of the cycle. Let y, z be

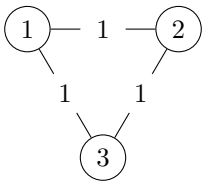
$$y_e = \begin{cases} x^*_e + \varepsilon & \text{if } e \in \{e_1, e_3, e_5, \ldots, e_{2k-1}\} \\ x^*_e - \varepsilon & \text{if } e \in \{e_2, e_4, e_6, \ldots, e_{2k}\} \\ x^*_e & \text{otherwise} \end{cases} \qquad z_e = \begin{cases} x^*_e - \varepsilon & \text{if } e \in \{e_1, e_3, e_5, \ldots, e_{2k-1}\} \\ x^*_e + \varepsilon & \text{if } e \in \{e_2, e_4, e_6, \ldots, e_{2k}\} \\ x^*_e & \text{otherwise} \end{cases}$$

Notice that the degree constraints are still satisfied by y and z as we are alternating between increasing and decreasing the edge values in a cycle of even length. Hence, to ensure feasibility, we need to choose such a small ε so as to guarantee that all y_e and z_e are in $[0,1]$. For example $\varepsilon = \min\{x^*_e, (1-x^*_e) : e \in E_f\}$ gives that both y and z are feasible. Now one can easily see that $x^* = \frac{1}{2}(y+z)$ which contradicts the assumption that x^* is an extreme point.

Because of the above claim, the polytope

$$P = \left\{ x : \sum_{b:(a,b)\in E} x_{(a,b)} = 1 \quad a \in A, \quad \sum_{a:(a,b)\in E} x_{(a,b)} = 1 \quad b \in B, \quad x_{(a,b)} \geq 0 \quad (a,b) \in E \right\}$$

is called the *bipartite perfect matching polytope*. Also, it says that we can solve the maximum weight bipartite matching problem by simply solving the above linear program.
Unfortunately, the degree-constraints are not sufficient for general graphs which can be seen by the following example:



Here, the solution that sets $x_e = 1/2$ for every edge e gives the optimal value $3/2$, but any integral matching has only value 1 (and the graph does not even have a perfect matching).

2.2.2 Bipartite Vertex Cover

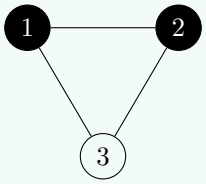
Here, we consider minimum weight vertex cover on bipartite graphs. The definition of the vertex cover problem is:

Definition 2.2.1: Min Weight Vertex Cover

Given a graph $G = (V, E)$ with node-weights $w : V \to \mathbb{R}$ find a vertex cover C (i.e., $e \cap C \neq \emptyset$ for all $e \in E$) that minimizes $w(C) = \sum_{v \in C} w(v)$.

Example 2.2.1

The minimum weight vertex cover (depicted in black) of the following graph is 3 (obtained by taking the vertices of weight 1 and 2).



Similarly to the case of maximum weight bipartite perfect matching, we formulate a linear program for the vertex cover problem. Here, we have a variable x_v for each vertex v with the intended meaning that it takes value 1 if that vertex is in the vertex cover. The constraints say that each edge needs to be covered. The LP can be formulated as follows:

$$\begin{aligned} \text{Minimize} \quad & \sum_{v \in V} x_v w(v) \\ \text{Subject to} \quad & x_u + x_v \geq 1 \quad \forall \{u, v\} \in E \\ & 0 \leq x_v \leq 1 \quad \forall v \in V \end{aligned}$$

We have the following structural result for bipartite graphs:

Claim 2.2.2

For bipartite graphs, any extreme point to the vertex cover LP is integral.

: Consider an extreme point x^* and let $V_f = \{v : 0 < x^*_v < 1\}$ be those vertices with fractional values in x^*. Suppose toward contradiction that $V_f \neq \emptyset$. Further, let A, B be the bipartition of V and let $A_f = V_f \cap A$ and $B_f = V_f \cap B$ be the fractional vertices in A and B, respectively. As for bipartite matchings, we reach a contradiction by defining feasible solutions y and z so that $x^* = \frac{1}{2}(y + z)$ which contradicts that x^* is an extreme point. Let $\varepsilon = \min\{x_v, (1 - x_v) : v \in A_f \cup B_f\}$ and define y and z by

$$y_v = \begin{cases} x^*_v + \varepsilon & \text{if } v \in A_f \\ x^*_v - \varepsilon & \text{if } v \in B_f \\ x^*_v & \text{otherwise} \end{cases} \qquad z_v = \begin{cases} x^*_v - \varepsilon & \text{if } v \in A_f \\ x^*_v + \varepsilon & \text{if } v \in B_f \\ x^*_v & \text{otherwise} \end{cases}$$

Clearly, we have that $x^* = \frac{1}{2}(y + z)$. It remains to verify that y and z are feasible solutions. Let us verify y (the argument that z is feasible is the same). By the selection of ε, we have that $0 \leq y_v \leq 1$ for all $v \in V$. We now need to verify that the constraint for every edge is satisfied. Consider an edge $\{a, b\} \in E$ in the bipartite graph. We need to verify that $y_a + y_b \geq 1$. If $x_a = 1$ (or $x_b = 1$), then we have that $y_a = 1$ (or $y_b = 1$) and so the constraint holds. Otherwise $0 < x_a, x_b < 1$ and so $a \in A_f$ and $b \in B_f$. This in turn implies that y is feasible since

$$y_a + y_b = (x_a + \varepsilon) + (x_b - \varepsilon) = x_a + x_b \geq 1,$$

where the last inequality holds because x^* is a feasible solution.

The above structural result says that we can solve minimum weighted vertex cover on bipartite graphs by simply solving the above linear program to find an optimal extreme point. It does not work for general graphs which can be seen by considering the triangle (as for matchings). In contrast to matchings, the vertex cover problem on general graphs is an NP-hard problem and we do not expect to have efficient algorithms for it.

2.3 Duality

2.3.1 Intuition

Consider the following linear program:

$$\begin{array}{lll} \text{Minimize} & 7x_1 + 3x_2 & \\ \text{Subject to:} & x_1 + x_2 \geq 2, & /\cdot y_1 \\ & 3x_1 + x_2 \geq 4, & /\cdot y_2 \\ & x_1, x_2 \geq 0. & \end{array}$$

We are looking for the optimal solution OPT to this LP. To find the solution we may ask two types of questions (to find the upper and lower bound on OPT). Is there a solution of cost ≤ 10? (Is OPT ≤ 10?) An answer to this type of questions is quite simple; we just find a feasible solution to the LP with objective function ≤ 10 e.g. $x_1 = x_2 = 1$. Is there no better solution? (Is OPT ≥ 10?) Observe that we can bound the objective function value using constraints that every feasible solution satisfies. From the first constraint we get

$$7x_1 + 3x_2 \geq x_1 + x_2 \geq 2.$$

Thus OPT ≥ 2. Similarly from the second constraint we get OPT ≥ 4. To make a better lower bound we will need to be more clever. Let's take a linear combination of the constraints with coefficients y_1, y_2 correspondingly. y_1, y_2 should be non-negative because multiplying a constraint by negative number would flip the inequality. By taking $y_1 = 1$, $y_2 = 2$ we obtain

$$7x_1 + 3x_2 \geq (x_1 + x_2) + 2(3x_1 + x_2) \geq 2 + 2 \cdot 4 = 10.$$

For each constraint of the given LP we associate a dual variable y_i denoting the weight of the i-th constraint. What kind of variables can we take to get a valid lower bound? How should we pick coefficients to maximize lower bound for OPT? First of all we are interested in lower-bounding the objective function. Thus linear combination of primal constraints cannot exceed primal objective function. As mentioned earlier $y_i \geq 0$. In this way we get the following (dual) linear program for y_1, y_2:

$$\begin{array}{lll} \text{Maximize} & 2y_1 + 4y_2 & \text{(lower bound as tight as possible)} \\ \text{Subject to:} & y_1 + 3y_2 \leq 7, & \text{(coefficient of } x_1\text{)} \\ & y_1 + y_2 \leq 3, & \text{(coefficient of } x_2\text{)} \\ & y_1, y_2 \geq 0. & \end{array}$$

Let's try now to formalize this approach.

2.3.2 General case

Consider the following linear program with n variables x_i for $i \in [1, n]$ and m constraints:

$$\begin{array}{ll} \text{Minimize} & \displaystyle\sum_{i=1}^{n} c_i x_i \\ \text{Subject to:} & \displaystyle\sum_{i=1}^{n} A_{ji} x_i \geq b_j \quad \forall j = 1, \ldots, m, \\ & x \geq 0. \end{array}$$

Then, the dual program has m variables y_j for $j \in [1, m]$ and n constraints:

$$\begin{array}{ll} \text{Maximize} & \displaystyle\sum_{j=1}^{m} b_j y_j \\ \text{Subject to:} & \displaystyle\sum_{j=1}^{m} A_{ji} y_j \leq c_i \quad \forall i = 1, \ldots, n, \\ & y \geq 0. \end{array}$$

Each variable y_j in the dual program corresponds to the weight of one of constraints from the primal LP. We have n constraints in the dual, one for every primal variable x_i.

Note:

We showed how to produce dual program for minimization problem. Similar approach works also for maximization problems. Also every maximization problem can be reduced to a minimization one. We replace x_i with $-x_i$ in constraints and objective function obtaining minimization problem.

Note:

One can verify that if we take the dual of the dual problem, we get back to the primal problem, as we should expect. Finding the dual linear program is an automatic procedure.

2.3.3 Duality Theorems

Let's focus now on the solutions of both LPs. In our example optimal solutions to primal and dual problems coincided. We now present two theorems that connect primal and dual solutions.

Theorem 2.3.1 Weak Duality

If x is primal-feasible (meaning that x is a feasible solution to the primal problem) and y is dual-feasible, then

$$\sum_{i=1}^{n} c_i x_i \;\geq\; \sum_{j=1}^{m} b_j y_j.$$

: Let's rewrite the right hand side

$$\sum_{j=1}^{m} b_j y_j \;\leq\; \sum_{j=1}^{m}\sum_{i=1}^{n} A_{ji} x_i y_j \;=\; \sum_{i=1}^{n}\Big(\sum_{j=1}^{m} A_{ji} y_j\Big) x_i \;\leq\; \sum_{i=1}^{n} c_i x_i.$$

Here we used the fact that $x, y \geq 0$ for the inequalities.

This theorem tells us that every dual-feasible solution is a lower bound to any primal solution. This is intuitive: every primal feasible solution satisfies primal constraints, dual feasible solution gives us a way of lower bounding primal solution using primal constraints. Moreover from Weak Duality we can conclude that optimal solution to primal program is lower bounded by optimal solution to dual program. In fact optimal solutions to primal and duals linear programs coincide, leading to the following theorem.

Theorem 2.3.2 Strong Duality

If x is an optimal primal solution and y is an optimal dual solution, then

$$\sum_{i=1}^{n} c_i x_i = \sum_{j=1}^{m} b_j y_j.$$

Furthermore, if the primal problem is unbounded, then the dual problem is infeasible and analogously if the dual is unbounded, the primal is infeasible.

We omit the proof of strong duality for now but I encourage you to understand one of the many proofs!

2.3.4 Example: Maximum cardinality matching and Vertex Cover on Bipartite Graphs

Let $G = (A \cup B, E)$ be a bipartite graph and let M be a matching. Let x_e be a variable corresponding to taking edge $e \in M$. We want to maximize the cardinality of M while assuring that every vertex has at most one neighboring edge belonging to M. Writing those conditions in a form of LP gives us:

$$\begin{aligned}
\text{Maximize} \quad & \sum_{e \in E} x_e \\
\text{Subject to:} \quad & \sum_{e=(a,b)\in E} x_e \leq 1 \quad \forall a \in A, \\
& \sum_{e=(a,b)\in E} x_e \leq 1 \quad \forall b \in B, \\
& x_e \geq 0.
\end{aligned}$$

Thus the dual program looks like this:

$$\begin{aligned}
\text{Minimize} \quad & \sum_{v \in A \cup B} y_v \\
\text{Subject to:} \quad & y_a + y_b \geq 1 \quad \text{for } (a,b) \in E, \\
& y_v \geq 0.
\end{aligned}$$

One can easily notice that this LP is vertex cover relaxation. By weak-duality, we have that $|M| \leq |C|$ for any matching M and vertex cover C. Moreover, since both the primal and the dual are integral for bipartite graphs, strong LP-duality implies König's theorem 1.3.1.

Another well-known duality that is also a special case of LP-duality is the max-flow = min-cut theorem.

2.3.5 Complementarity Slackness

Strong duality gives an important relationship between primal and dual optimal solutions.

Theorem 2.3.3

Let $x \in \mathbb{R}^n$ be a feasible solution to the primal and let $y \in \mathbb{R}^m$ be a feasible solution to the dual. Then

$$x, y \text{ are both optimal solutions} \iff \begin{cases} x_i > 0 \;\Rightarrow\; c_i = \sum_{j=1}^{m} A_{ji} y_j & \forall i = 1, \ldots, n, \\ y_j > 0 \;\Rightarrow\; b_j = \sum_{i=1}^{n} A_{ji} x_i & \forall j = 1, \ldots, m. \end{cases}$$

$:$ We will apply the strong duality theorem to the weak duality theorem proof.

$\Rightarrow$ Let x be the optimal primal solution. From the weak duality theorem proof, we have that

$$\sum_{j=1}^{m} b_j y_j \;\leq\; \sum_{j=1}^{m}\sum_{i=1}^{n} A_{ji} x_i y_j \;=\; \sum_{i=1}^{n} \Big(\sum_{j=1}^{m} A_{ji} y_j \Big) x_i \;\leq\; \sum_{i=1}^{n} c_i x_i. \tag{2.1}$$

Here we used the fact that $x, y \geq 0$. On the other hand by the strong duality theorem

$$\sum_{j=1}^{m} b_j y_j = \sum_{i=1}^{n} c_i x_i.$$

So in (2.1) there are equalities everywhere. Thus

$$\sum_{i=1}^{n} c_i x_i = \sum_{i=1}^{n} \Big(\sum_{j=1}^{m} A_{ji} y_j \Big) x_i \;\Rightarrow\; c_i x_i = \Big(\sum_{j=1}^{m} A_{ji} y_j \Big) x_i \quad \text{for } i = 1, \ldots, n.$$

And finally for every x_i, $i = 1, \ldots, n$:

$$x_i \neq 0, \quad c_i x_i = \Big(\sum_{j=1}^{m} A_{ji} y_j \Big) x_i \;\Rightarrow\; c_i = \Big(\sum_{j=1}^{m} A_{ji} y_j \Big).$$

$\Leftarrow$ Similarly to the previous part we know that:

$$x_i c_i = \Big(\sum_{j=1}^{m} A_{ji} y_j\Big) x_i \qquad \forall i = 1, \ldots, n,$$

$$y_j b_j = \Big(\sum_{i=1}^{n} A_{ji} x_i\Big) y_j \qquad \forall j = 1, \ldots, m.$$

Thus

$$\sum_{j=1}^{m} b_j y_j = \sum_{j=1}^{m} \sum_{i=1}^{n} A_{ji} x_i y_j = \sum_{i=1}^{n} \Big(\sum_{j=1}^{m} A_{ji} y_j\Big) x_i = \sum_{i=1}^{n} c_i x_i.$$

The above equality is equivalent to x, y being optimal solutions to the primal and the dual linear programs, respectively. Indeed for feasible solution $x^\star$ to the primal we have by weak duality

$$\sum_{i=1}^{n} c_i x_i^\star \;\geq\; \sum_{j=1}^{m} b_j y_j = \sum_{i=1}^{n} c_i x_i.$$

Thus x is an optimal solution to the primal program and similarly y is an optimal solution to the dual. ☺

2.4 Hungarian Method

2.4.1 Duality and Complementarity Slackness of Min-Cost Perfect Matching

Let $G = (A \cup B, E)$ be a bipartite weighted graph with edge-costs $c : E \to \mathbb{R}$. We wish to find a perfect matching M of minimum cost $\sum_{e \in M} c(e)$.
To write down the linear program for this problem, we have a variable x_e for every edge e, with the intended meaning that x_e takes value 1 if e is in the matching and 0 otherwise. In a perfect matching, every vertex is adjacent to exactly one edge of the matching. This leads to the following linear program:

$$\begin{aligned}
\text{Minimize} \quad & \sum_{e \in E} c(e) x_e \\
\text{Subject to:} \quad & \sum_{b \in B : (a,b) \in E} x_{ab} = 1 \quad \forall a \in A, \\
& \sum_{a \in A : (a,b) \in E} x_{ab} = 1 \quad \forall b \in B, \\
& x_e \geq 0 \quad \forall e \in E.
\end{aligned}$$

In a previous lecture, we saw that any extreme point of the above linear program is integral. Hence, we could solve the min-cost perfect matching problem by simply solving the above linear program, finding an extreme-point solution. However, this may be unnecessarily inefficient. Instead, we will see how to use LP duality to, basically, reduce this (weighted) problem to that of finding a perfect matching in an unweighted graph. (A problem that we already saw how to solve using augmenting paths.)

Obtaining the dual. To take the dual of the above program, let us first write it in the same form as last lecture, i.e. a minimization problem with $\geq$ and nonnegativity constraints. We can do this by replacing each equality by two inequalities. This gives us the following equivalent linear program:

$$\begin{aligned}
\text{Minimize} \quad & \sum_{e \in E} c(e) x_e \\
\text{Subject to:} \quad & \sum_{b \in B : (a,b) \in E} x_{ab} \geq 1 \quad \forall a \in A, \\
& -\sum_{b \in B : (a,b) \in E} x_{ab} \geq -1 \quad \forall a \in A, \\
& \sum_{a \in A : (a,b) \in E} x_{ab} \geq 1 \quad \forall b \in B, \\
& -\sum_{a \in A : (a,b) \in E} x_{ab} \geq -1 \quad \forall b \in B, \\
& x_e \geq 0 \quad \forall e \in E.
\end{aligned}$$

For each $a \in A$, we then associate a variable u_a^+ to the first constraint for a ($\sum_{b \in B : (a,b) \in E} x_{ab} \geq 1$) and u_a^- to the second constraint for a ($-\sum_{b \in B : (a,b) \in E} x_{ab} \geq -1$). Similarly, we have variables v_b^+ and v_b^- for each $b \in B$. These variables take the same role as the y-variables: the dual has a variable for each constraint in the primal. We now have that the dual equals

$$\begin{aligned}
\text{Maximize} \quad & \sum_{a \in A} (u_a^+ - u_a^-) + \sum_{b \in B} (v_b^+ - v_b^-) \\
\text{Subject to:} \quad & (u_a^+ - u_a^-) + (v_b^+ - v_b^-) \leq c(e) \quad \forall e = (a,b) \in E, \\
& u_a^+, u_a^-, v_b^+, v_b^- \geq 0 \quad \forall a \in A,\ b \in B.
\end{aligned}$$

Complementarity Slackness. Complementarity slackness tells us that if $x, (u^+, u^-, v^+, v^-)$ are feasible, then they are both optimal if and only if the following holds:

$$\begin{aligned}
&x_e > 0 \Rightarrow (u_a^+ - u_a^-) + (v_b^+ - v_b^-) = c(e) && \forall e = (a,b) \in E,\\
&u_a^+ > 0 \Rightarrow \sum_{b \in B:(a,b)\in E} x_{ab} = 1 && \forall a \in A,\\
&u_a^- > 0 \Rightarrow -\sum_{b \in B:(a,b)\in E} x_{ab} = -1 && \forall a \in A,\\
&v_b^+ > 0 \Rightarrow \sum_{a \in A:(a,b)\in E} x_{ab} = 1 && \forall b \in B,\\
&v_b^- > 0 \Rightarrow -\sum_{a \in A:(a,b)\in E} x_{ab} = -1 && \forall b \in B.
\end{aligned}$$

We have thus obtained the dual and the complementarity slackness conditions in an "automatic" way.

Simplifying the notation. Before continuing, we would like to make an observation that will simplify our description and notation. Namely, in the dual that we obtained, the variables u_a^+ and u_a^- always appear only as part of the expression $(u_a^+ - u_a^-)$. Although we have $u_a^+, u_a^- \geq 0$, this expression can take any value (positive and negative). We can thus for every $a \in A$ replace $(u_a^+ - u_a^-)$ by a new variable $u_a \in \mathbb{R}$ whose value is not constrained to be nonnegative (it can take both positive and negative values). We do the same for each $b \in B$ and replace $(v_b^+ - v_b^-)$ by v_b.[1] We can thus write the primal and the dual as follows:

$$\begin{aligned}
\text{Minimize} \quad & \sum_{e \in E} c(e) x_e \\
\text{Subject to:} \quad & \sum_{b \in B:(a,b)\in E} x_{ab} = 1 \quad \forall a \in A, \\
& \sum_{a \in A:(a,b)\in E} x_{ab} = 1 \quad \forall b \in B, \\
& x_e \geq 0 \quad \forall e \in E.
\end{aligned}$$

and

$$\begin{aligned}
\text{Maximize} \quad & \sum_{a \in A} u_a + \sum_{b \in B} v_b \\
\text{Subject to:} \quad & u_a + v_b \leq c(e) \quad \forall e = (a,b) \in E.
\end{aligned}$$

With this simplified notation, complementarity slackness gives us that if $x, (u, v)$ are feasible, then they are both optimal if and only if the following holds:

$$\begin{aligned}
&x_e > 0 \Rightarrow u_a + v_b = c(e) && \forall e = (a,b) \in E,\\
&u_a \neq 0 \Rightarrow \sum_{b \in B:(a,b)\in E} x_{ab} = 1 && \forall a \in A,\\
&v_b \neq 0 \Rightarrow \sum_{a \in A:(a,b)\in E} x_{ab} = 1 && \forall b \in B.
\end{aligned}$$

As a final simplification, observe that the last two conditions always hold since they follow immediately from the fact that x is primal-feasible (this is because in this LP we have equalities instead of inequalities). We can thus summarize the above with the following:

[1] Note that our new LP is in fact equivalent to the old one. Given a feasible solution to the old LP, we can obtain a feasible solution to the new LP by setting the values for the new variables as $u_a := u_a^+ - u_a^-$ (and similarly for v). Since we do the same in the objective function, the objective value remains unchanged. We can also go back and obtain a solution of the old LP from a solution of the new LP as follows: if $u_a \geq 0$, set $u_a^+ = u_a$ and $u_a^- = 0$; otherwise set $u_a^+ = 0$ and $u_a^- = -u_a$ (and similarly for v).

Also note that the new LP is what we would have obtained if we had multiplied each equality constraint $\sum_{b \in B:(a,b)\in E} x_{ab} = 1$ by an unconstrained dual variable $u_a \in \mathbb{R}$, rather than rewriting this equality constraint as two inequality constraints and then multiplying each by a nonnegative dual variable (u_a^+ or u_a^-). (And similarly for v.)

Lenma 2.4.1

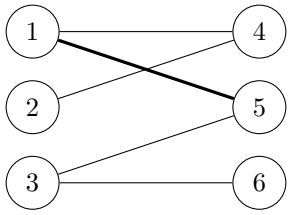
A perfect matching M is of minimum cost iff there is a feasible dual solution u, v such that $u_a + v_b = c(e)$ for every $e = (a, b) \in M$.

2.4.2 The Hungarian Algorithm

We start with some intuition before we give the formal description of the algorithm.

Intuition

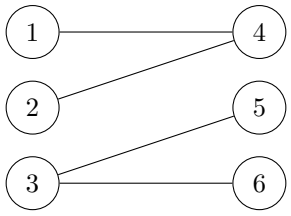
Consider the following instance of the min-cost perfect matching problem:



The thin edges have cost 1, whereas the thick edge has cost 2. The Hungarian algorithm will use Lemma 2.4.1 in the following way: we will maintain a dual solution u, v that is feasible at all times. Then, for a fixed dual solution, the lemma tells us that our perfect matching is only allowed to contain edges that are *tight*, i.e., edges $e = (a, b)$ for which $u_a + v_b = c(e)$. This reduces our problem to finding any perfect matching in the subgraph consisting only of tight edges, i.e., in the graph (V, E_0) where $E_0 = \{e = (a, b) \in E : u_a + v_b = c(e)\}$. Intuitively, we have thus reduced our weighted problem to an unweighted one!
Let us return to our example. We initialize our procedure with the trivial dual solution

$$v_b = 0, \qquad u_a = \min_{b \in B} c_{ab}.$$

(We could have also started from $u = v = 0$.) So in the example, $v_4 = v_5 = v_6 = 0$ and $u_1 = u_2 = u_3 = 1$. The set E_0 of tight edges is thus:



We then try to find a perfect matching in this graph using e.g. the augmenting path algorithm we saw in Lecture 3. However, the considered graph has no perfect matching! This is because we started with a poor lower bound (dual solution). So we will use the fact that there is no perfect matching to improve our dual solution. We make use of Hall's condition:

Theorem 2.4.1 Hall's Theorem

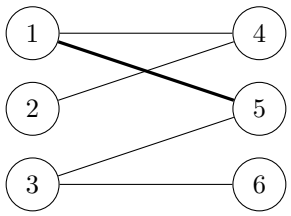
An n-by-n bipartite graph $G = (A \cup B, E_0)$ has a perfect matching if and only if $|S| \leq |N(S)|$ for all $S \subseteq A$, where:

$$N(S) = \{b \in B : \text{there is an } a \in S \text{ such that } (a, b) \in E_0\}$$

: Lasciata a DarioIlDistruttore

Here $N(S)$ denotes the neighborhood of the vertices in S.
In the above example, we have $S = \{1, 2\}$ and $N(S) = \{4\}$, which violates Hall's condition (and thus there is no perfect matching). We now use the set S to improve our dual lower bound.

We gradually increase u_a for every $a \in S$ and at the same time decrease v_b for $b \in N(S)$ at the same rate. Let us see what happens to all edges in E_0. Notice that the tight edges between S and $N(S)$ will remain tight. Similarly, the tight edges between $A \setminus S$ and $B \setminus N(S)$ will remain tight. Any tight edges between $A \setminus S$ and $N(S)$ will stop being tight. Finally, by definition, there are no edges from S to $B \setminus N(S)$ initially. We continue to gradually change the dual solution until some such edge becomes tight.
In the above example, this will result in updating $u_1 = u_2 = 2$ and $v_4 = -1$. We have thus increased two variables by one unit and decreased one variable by one unit. In total, the dual lower bound was thus increased by one. The set E_0 of tight edges with respect to the new dual solution is now



Our (augmenting-path) algorithm will now find a perfect matching in this graph, which is optimal by Lemma 2.4.1.

In summary, our algorithm always maintains a dual feasible solution. We then solve the unweighted perfect matching problem on the edges allowed by Lemma 2.4.1. In the case of failure, we update the dual lower bound to a strictly better lower bound and repeat.

Formal description

Idea: Maintain a feasible dual solution (u, v). Try to construct a feasible primal solution that satisfies complementarity slackness (Lemma 2.4.1) and is thus optimal.

Algorithm:

- **Initialization:** $v_b = 0,\ u_a = \min_{b \in B} c_{ab}$.
- **Iterative step:**
  - consider $G_0 = (A \cup B, E_0)$ where $E_0 = \{e = (a, b) \in E : u_a + v_b = c(e)\}$ (E_0 is the set of all tight edges);
  - find a maximum-cardinality matching in G_0:
    - if it is a perfect matching, then we are done (this is a primal feasible solution and it satisfies complementarity slackness, because we consider only the edges in E_0 and all of them satisfy slackness by construction),
    - otherwise the algorithm finds a set $S \subseteq A$ s.t. $|S| > |N(S)|$ (which is guaranteed to exist by Hall's theorem)
  - we can choose a small $\varepsilon > 0$ and improve the dual solution:

$$u'_a = \begin{cases} u_a + \varepsilon & \text{if } a \in S, \\ u_a & \text{if } a \notin S, \end{cases} \qquad v'_b = \begin{cases} v_b - \varepsilon & \text{if } b \in N(S), \\ v_b & \text{if } b \notin N(S), \end{cases}$$

    which remains dual feasible because:
    - edges in $S \times N(S)$ are unchanged ($+\varepsilon - \varepsilon$),
    - edges in $(A \setminus S) \times (B \setminus N(S))$ are unchanged,
    - edges in $(A \setminus S) \times N(S)$ are decreased by ε,
    - edges in $S \times (B \setminus N(S))$ are increased by ε but they were not tight (they were not in E_0),
  - the dual value increases by $(|S| - |N(S)|)\varepsilon$;
  - to make as much progress as possible (and to get a new tight edge), we should choose ε as large as possible for which the dual remains feasible, that is

$$\varepsilon = \min_{e=(a,b) \in S \times (B \setminus N(S))} c(e) - u_a - v_b > 0.$$

The above algorithm can be implemented to run in time $O(n^3)$. (This is quite non-trivial to see. It is somewhat easier to implement in $O(n^4)$.)

An alternative proof that the bipartite perfect matching polytope is integral

We have shown that for any cost function c we can obtain a minimum-cost perfect matching and it will be integral. By carefully choosing the cost function, one can make any extreme point of the polytope to be the unique optimum solution to the minimum-cost perfect matching problem. This shows that all extreme points are integral, i.e., it is an integral polytope. See Figure 2.1 for an example.

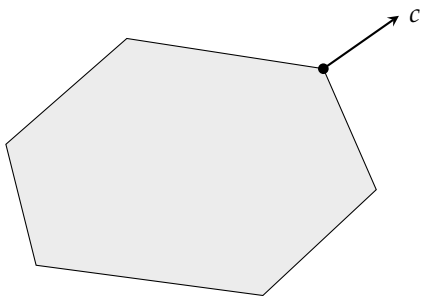


Figure 2.1: Example of a perfect matching polytope and a cost function vector c for the highlighted vertex.

Chapter 3

Approximation Algorithms

3.1 When Relaxing Integrality is Not for Free: Approximation Algorithms

Thousands (or millions :)) of optimization problems have been proved to be NP-hard, which means that finding optimal solutions for them is very likely to be intractable. More specifically, unless $\mathsf{P} = \mathsf{NP}$, there is no algorithm for an NP-hard optimization problem that satisfies all of the following three desiderata:

1. The algorithm is *efficient* (runs in polynomial time).
2. The algorithm is *reliable* (works for any input instance).
3. The algorithm finds an optimal solution (*optimality*).

Therefore, when confronted which such problems, we need to relax one of the above conditions when dealing with NP-hard optimization problems. If we relax reliability, then we get *heuristics* that are designed to work well on instances that commonly appear in practice. If we relax the third condition, we obtain *approximation algorithms*. The notion of approximation algorithms allows us to obtain a more fine-grained picture of the difficulty of NP-hard optimization problems: some problems turn out to have very good approximations, whereas others resist such attempts.
The formal definition of approximation algorithms is as follows.

Definition 3.1.1: α-approximation algorithm

An α-approximation algorithm for a given optimization problem is an algorithm that runs in polynomial time and outputs a solution S such that:

- $\dfrac{\text{cost}(S)}{\text{cost(Optimal solution)}} \leq \alpha$ if the problem is a minimization problem,
- $\dfrac{\text{profit}(S)}{\text{profit(Optimal solution)}} \geq \alpha$ if the problem is a maximization problem.

It is clear that we will have $\alpha \geq 1$ for minimization problems and $\alpha \leq 1$ for maximization problems. Moreover, if $\alpha = 1$, then we have an efficient exact algorithm (and the problem is in P). The goal when designing approximation algorithms is to achieve a guarantee α that is as close to 1 as possible. We now introduce a general framework for using linear programming in the design of approximation algorithms followed by an application: vertex cover.

3.2 Using Linear Programming to Design Approximation Algorithms

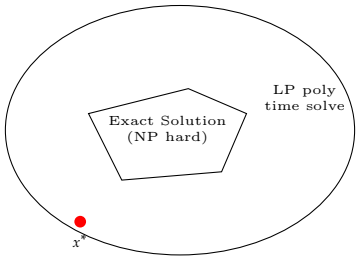
Consider a minimization problem. When considering the definition of approximation algorithms, it seems very hard, even for a specific instance of the problem, to analyze the approximation guarantee $\frac{\text{cost}(S)}{\text{cost(Optimal solution)}}$ of the algorithm on the instance, as we are comparing ourselves with an optimal solution (that is most likely very

hard to compute and does not possess any nice structure). The solution to this is to compare ourselves with a lower bound on the optimum. Indeed, we then have that

$$\frac{\text{cost}(S)}{\text{cost(Optimal solution)}} \leq \frac{\text{cost}(S)}{\text{lower bound on opt}} \leq \alpha.$$

From the above, it is clear that we need a good lower bound to be able to claim a good guarantee α. To obtain such a lower bound (and to design the approximation algorithm), linear programming is super handy. A popular framework is as follows:

1. Give an exact formulation of the problem as Integer LP – usually with binary variables ($x_i \in \{0,1\}$).
2. Relax to LP – $x_i \in [0,1]$



3. Solve LP to get a optimal solution x^* to the LP which is a lower (upper) bound on the optimal solution to Integer LP and thus the original problem. Then somehow round x^* to an integral solution "without losing too much" (which will determine the guarantee α).

We now use the above framework for the vertex cover problem, and then introduce the concept of *integrality gap*.

3.2.1 Vertex Cover

Here, we apply the framework to get an approximation algorithm for the *Vertex Cover* problem. First recall the definition:

Definition 3.2.1: Vertex Cover (VC) Problem

Given a graph $G = (V, E)$ and a weight function on the vertices $w : V \to \mathbb{R}_+$, output a set $C \subseteq V$ of minimum weight such that for all $\{u, v\} \in E$, $u \in C$ or $v \in C$.

First, we define an ILP solving VC: For all $v \in V$, we introduce a variable x_v, which is 1 if $v \in C$, and 0 otherwise. The objective function is

$$\min \sum_{v \in V} w(v) x_v$$

and for each $\{u, v\} \in E$ we introduce the constraint $x_u + x_v \geq 1$. This ensures that for each edge, at least one endpoint is in C. Additionally, we require that for each $v \in V$ we have $x_v \in \{0, 1\}$. Note that at this point our Integer LP formulation is exactly equivalent to the original problem.
Second, we relax the ILP to an LP by allowing that $x_v \in [0, 1]$. Actually, it's sufficient to require that $x_v \geq 0$, because having an x_v greater than 1 will not satisfy any additional constraint, but only increase the value of the objective function, so this will not happen in an optimal solution.
Third, we have to do the rounding: Suppose we've solved the LP and got an optimal solution x^*. We will return $C = \{v \in V : x_v^* \geq \frac{1}{2}\}$ as our solution to VC.

Claim 3.2.1

C is a feasible solution.

: Consider any edge $\{u, v\}$ and its constraint. Since $x_u^* + x_v^* \geq 1$, at least one of x_u^*, x_v^* is $\geq \frac{1}{2}$ and thus in C. ☺

Claim 3.2.2

The weight of C is at most twice the value of the optimal solution of VC.

: We have

$$\sum_{v \in C} w(v) = \sum_{v \in V : x_v^* \geq \frac{1}{2}} w(v) \leq \sum_{v \in V : x_v^* \geq \frac{1}{2}} 2x_v^* w(v) \leq \sum_{v \in V} 2x_v^* w(v) = 2 \sum_{v \in V} x_v^* w(v) = 2\mathrm{LP}_{\mathrm{OPT}} \leq 2\mathrm{VC}_{\mathrm{OPT}}$$

where the first inequality holds because $2x_v^* \geq 1$.

So, we have designed a 2-approximation algorithm for Vertex Cover. This is a simple algorithm using linear programming. To appreciate the power of the framework, I challenge you to find a 2-approximation algorithm for Vertex Cover (with node-weights) without the use of LPs. (It is possible but quite difficult.)

3.2.2 Integrality Gap

The notion of the *integrality gap* allows us to bound the power of our linear programming relaxation. Let $\mathcal{I}$ be the set of all instances of a given problem. In the case of a minimization problem, the integrality gap g is defined as

$$g = \max_{I \in \mathcal{I}} \frac{\mathrm{OPT}(I)}{\mathrm{OPT}_{LP}(I)}.$$

As an example, suppose $g = 2$, and that LP found that $\mathrm{OPT}_{LP} = 70$. Then, since our problem instance might be the one which maximizes the expression for g, all we can guarantee is that $\mathrm{OPT}(I) \leq 2 \cdot \mathrm{OPT}_{LP}(I) = 140$, so it's not possible to find an approximation algorithm (using only this linear programming relaxation) that approximates better than within a factor of $g = 2$.

Integrality Gap for Vertex Cover

Claim 3.2.3

The integrality gap for vertex cover is at least $2 - \frac{2}{n}$.

: Consider the complete graph on n vertices. We have $\mathrm{OPT} = n - 1$, because if there are 2 vertices that we don't choose, the edge between them is not covered. However, $\mathrm{LP}_{\mathrm{OPT}} \leq \frac{n}{2}$, because assigning $\frac{1}{2}$ to each vertex is a feasible solution of cost $\frac{n}{2}$, so the optimum can only be smaller. Now,

$$g \geq \frac{n-1}{\frac{n}{2}} = 2 - \frac{2}{n}.$$

We remark that our 2-approximation algorithm for vertex cover implies that the integrality gap is at most 2.

3.3 Set Cover via Randomized Rounding

Let us now apply the framework to the Set Cover problem. It can be seen as a generalization of the vertex cover problem and its definition is as follows:

Definition 3.3.1: Set Cover Problem

Given a universe $U = \{e_1, e_2, \ldots, e_n\}$, and a family of subsets $\mathcal{T} = \{S_1, S_2, \ldots, S_m\}$ and a cost function $c : \mathcal{T} \to \mathbb{R}_+$, find a collection C of subsets of minimum cost that cover all elements.

As for vertex cover, we start by giving an exact Integer LP formulation. For each $i \in \{1, \ldots, m\}$, define x_i, which is 1 if $S_i \in C$, and 0 otherwise. The objective function is

$$\min \sum_{i=1}^{m} x_i \cdot c(S_i)$$

and for each element $e \in U$, we add the constraint $\sum_{S_i : e \in S_i} x_i \geq 1$. This ensures that each element is covered by at least one set in C. And for each x_i, we require that $x_i \in \{0, 1\}$ in the ILP. The LP relaxation is then obtained by replacing the boolean constraints $x_i \in \{0, 1\}$ by $x_i \in [0, 1]$.
Now suppose that each element belongs to at most f sets. Then, as in your exercise on vertex cover on k-uniform hypergraphs, we can do the following rounding: $C = \{S_i : x_i^* \geq \frac{1}{f}\}$. In each constraint, there's at least one x_i^* which is at least $\frac{1}{f}$, so each constraint is satisfied. Using the same reasoning as in the analysis of the vertex cover rounding, we can show that this approximation is within a factor of f.

3.3.1 A better approximation for Set Cover

If we introduce randomness and allow our algorithm to output non-feasible solutions with some small probability, we can get much better results (in expectation).
We use the same LP as in the previous section, and will run the following algorithm:

1. Solve the LP to get an optimal solution x^*.
2. Choose some positive integer constant d (we will see later how d affects the guarantees we get). Start with an empty result set C, and repeat step 3 $d \cdot \ln(n)$ times.
3. For $i = 1, \ldots, m$, add set S_i to the solution C with probability x_i^*, choosing independently for each set.

Now let us analyze what guarantees we can get:

Claim 3.3.1
The expected cost of all sets added in one execution of Step 3 is $\sum_{i=1}^{m} x_i^* c(S_i) = \mathrm{LP}_{\mathrm{OPT}}$.

:

$$\mathbb{E}[\text{rounded cost}] = \sum_{i=1}^{m} c(S_i) \Pr[S_i \text{ is added}] = \sum_{i=1}^{m} c(S_i) x_i^* = \mathrm{LP}_{\mathrm{OPT}}$$

From this, we can immediately derive

Corollary 3.3.1
The expected cost of C after $d \cdot \ln(n)$ executions of Step 3 is at most

$$d \cdot \ln(n) \cdot \sum_{i=1}^{m} c(S_i) x_i^* \;\leq\; d \cdot \ln(n) \cdot \mathrm{LP}_{\mathrm{OPT}} \;\leq\; d \cdot \ln(n) \cdot \mathrm{OPT}$$

Note that we have $\mathrm{LP}_{\mathrm{OPT}} \leq \mathrm{OPT}$ because LP is a relaxation of the original problem, so its optimum can only be better.
That sounds good, but we should also worry about feasibility:

Claim 3.3.2
The probability that a constraint remains unsatisfied after a single execution of Step 3 is at most $\frac{1}{e}$.

: Suppose our constraint contains k variables, and let us write it as $x_1 + x_2 + \cdots + x_k \geq 1$. Then,

$$\begin{aligned}
\Pr[\text{constraint unsat.}] &= \Pr[S_1 \text{ not taken}] \cdots \Pr[S_k \text{ not taken}] \\
&= (1 - x_1^*) \cdots (1 - x_k^*) \\
&\leq e^{-x_1^*} \cdots e^{-x_k^*} \qquad (3.1) \\
&= e^{-\sum_{i=1}^{k} x_i^*} \\
&\leq e^{-1} \qquad (3.2)
\end{aligned}$$

where (3) follows from the inequality $1 - x \leq e^{-x}$ and (4) from the fact that $\sum_i x_i^* \geq 1$.

Claim 3.3.3

The output C is a feasible solution with probability at least $1 - \frac{1}{n^{d-1}}$.

: Using claim, we find that the probability that a given constraint is unsatisfied after $d \cdot \ln(n)$ executions of step 3 is at most

$$\left(\frac{1}{e}\right)^{d \cdot \ln(n)} = \frac{1}{n^d}$$

and by union-bound, the probability that there exists any unsatisfied constraint is at most

$$n \cdot \frac{1}{n^d} = \frac{1}{n^{d-1}}$$

Now we have an expected value for the cost, and also a bound on the probability that an infeasible solution is output, but we still might have a bad correlation between the two: It could be that all feasible outputs have a very high cost, and all infeasible outputs have a very low cost.
The following claim deals with that worry.

Claim 3.3.4

The algorithm outputs a feasible solution of cost at most $4d \ln(n)\mathrm{OPT}$ with probability greater than $\frac{1}{2}$.

: Let μ be the expected cost, which is $d \ln(n) \cdot \mathrm{OPT}$ by corollary. We can upper-bound the bad event that the actual cost is very high: By Markov's inequality, we have $\Pr[\text{cost} > 4\mu] \leq \frac{1}{4}$. The other bad event that we have to upper bound is that the output is infeasible, and by claim, we know that this happens with probability at most $\frac{1}{n^{(d-1)}} \leq \frac{1}{n}$. Now in the worst case, these two bad events are completely disjoint, so the probability that no bad event happens is at least $1 - \frac{1}{4} - \frac{1}{n}$, and if we suppose that n is greater than 4, this probability is indeed greater than $\frac{1}{2}$.

We have thus designed a randomized $O(\log n)$-approximation algorithm for the set cover problem.
We remark that the used framework has the following general advantage (compared to worst-case guarantees): we can often get better per-instance guarantee than the general approximation factor: Suppose we have an instance where $\mathrm{LP}_{\mathrm{OPT}} = 100$, and our algorithm found a solution of cost 110. Since we know that $\mathrm{LP}_{\mathrm{OPT}} \leq \mathrm{OPT}$, we can say that our solution on this instance is at most 10% away from the optimal solution for this instance.

3.4 Integrality gap of the set cover LP

Consider the following instance of the Set Cover problem. For an even integer $d \geq 1$ let

$$U = \left\{x \in \{0,1\}^d : \sum_{i=1}^{d} x_i = d/2\right\},$$

i.e., the universe consists of all binary vectors of length d that have $d/2$ nonzeros. Let the collection $\mathcal{F}$ contain $m = d$ sets $S_1, \ldots, S_m$, defined by

$$S_i = \{x \in U : x_i = 1\}$$

for every $i = 1, \ldots, m$. All costs are 1.
We first give a feasible solution to the LP relaxation of Set Cover on the instance above with value bounded by 2. The LP relaxation of the set cover problem is the following:

$$\min_z \sum_{i=1}^{d} z_i, \quad \text{st.:} \quad \forall i \in [d] : z_i \in [0,1], \qquad \forall x \in U : \sum_{i:\, x \in S_i} z_i \geq 1$$

A solution to this with value 2 is to set every variable z_i to $2/d$. That way $\sum_i z_i = 2$ and all constraints are satisfied:

$$\sum_{i:\, x \in S_i} z_i = \sum_{i:\, x_i = 1} z_i = \sum_{i:\, x_i = 1} 2/d = \sum_{i=1}^{d} 2/d \cdot x_i = 1$$

Suppose we have any collection of $d/2$ sets $\mathcal{F}' \subseteq \mathcal{F}$. We can characterise $\mathcal{F}'$ as $\{S_i : i \in I\}$ for some $I \subseteq \{1, \ldots, d\}$ with $|I| = d/2$. Let us then define the vector x^* such that $x^*_i = 0$ for $i \in I$ and $x^*_i = 1$ for $i \notin I$. Then $x^* \in U$ but $x^* \notin \cup\mathcal{F}'$, thus proving that $\mathcal{F}'$ does not cover U. In this set cover problem the optimal integral solution is at least $d/2 + 1$, but the optimal fractional solution is at most 2. This is an $\Omega(d)$ integrality gap. Since the size of the universe, $|U| = \binom{d}{d/2} \leq 2^d$, this translates to an $\Omega(\log |U|)$ integrality gap.

Chapter 4

The Multiplicative Weights Algorithm

This section is basically taken verbatim from the excellent lecture notes[1] by Anupam Gupta available here (together with a lot of other interesting information): https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15859-f11/www/
Multiplicative weights is a retronym for the simple iterative rule that underlies modular, iterative algorithms for solving LPs and SDPs (semidefinite programs).[2] The Multiplicative Weights Algorithm is known by diferent names in the various fields where it was (re)discovered. Check out the survey by Arora, Hazan and Kale [3]; our discussion will be based on their treatment. Due to its broad appeal, we will consider multiplicative weights in more generality than is needed for solving LPs and SDPs. In this lecture, we'll introduce some strategies for playing a prediction game.

4.1 Naive Implementations

4.1.1 Warmup: Prediction with Expert Advice

The following sequential game is played between an omniscient Adversary and an Aggregator who is advised by N experts. Special cases of this game include predicting if it will rain tomorrow, or if the stock market will go up or down.

Strategy 1 Prediction with Expert Advice

1: **for** $t = 1 \dots T$ **do**
2: Each expert $i \in [N]$ advises either YES or NO
3: Aggregator predicts either YES or NO
4: Adversary, with knowledge of the expert advice and Aggregator's prediction, decides the YES/NO outcome.
5: Aggregator observes the outcome and suffers if his prediction was incorrect.
6: **end for**

Naturally, Aggregator wants to make as few mistakes as possible. Since the experts may be unhelpful and the outcomes may be capricious, Aggregator can hope only for a relative performance guarantee. In particular, Aggregator hopes to do as well as the best single expert in hindsight.[3] In order to do so, Aggregator must track which experts are helpful. We will consider a few tracking strategies. Almost every other aspect of the game – that advise is aggregated into a single value, that this value is binary, and even that the game is sequential – is not relevant; we will generalize or eliminate these aspects.
If there is a perfect expert, then an obvious strategy is to dismiss experts who are not perfect. With the remaining experts, take a majority vote. Whenever the Aggregator makes a mistake, at least half of the remaining experts

[1] This section corresponds to Lecture 16 in the course, which was scribed by Shiva Kaul
[2] See Lecture 17 in the course linked earlier
[3] The excess number of mistakes is called (external) *regret*

are dismissed, so Aggregator makes at most $\log N$ mistakes.[4] We can use the same strategy even when there isn't a perfect expert, if we restart after every expert has been eliminated. If the best expert has made M mistakes by time T, then Aggregator has restarted at most $M+1$ times, so it has made at most $(M+1)\cdot\log N$ mistakes. This bound is rather poor since it depends multiplicatively on M.

4.1.2 Fewer Mistakes with Weighted Majority

We may obtain an additive mistake bound by softening our strategy: instead of dismissing experts who erred, discount their advice. This leads to the Weighted Majority Algorithm of Littlestone and Warmuth [4]. Assign each expert i a weight $w_i^{(1)}$ initialized to 1. Then for every t, predict YES/NO based on the weighted majority vote and halve the mistaken experts' weights after observing the outcome. The game strategy then becomes:

Strategy 2 Prediction with Weighted Majority

1: Initialize $w^{(1)} = (w_1^{(1)}, \ldots, w_N^{(1)})$ to be a vector of 1's
2: **for** $t = 1 \ldots T$ **do**
3: Each expert $i \in [N]$ advises either YES or NO
4: Aggregator predicts either YES or NO based on a weighted majority vote using $w^{(t)}$
5: Adversary, with knowledge of the expert advice and Aggregator's prediction, decides the YES/NO outcome.
6: Aggregator observes the outcome and for every mistaken expert i, set $w_i^{(t+1)} \leftarrow w_i^{(t)}/2$
7: **end for**

Claim 4.1.1

For any sequence of outcomes, duration T, let M_{WM} be the number of mistakes that the Weighted Majority strategy makes, and M_i be the number of mistakes that expert i makes. Then

$$M_{\mathrm{WM}} \le 2.41 \cdot (M_i + \log N)$$

: Let

$$\Phi^{(t)} = \sum_{i\in[N]} w_i^{(t)}$$

be a 'potential' function. Observe that:

- By definition, we have $\Phi^{(1)} = N$
- Also by definition, $\left(\frac{1}{2}\right)^{M_i} \le \Phi^{(T+1)}$
- At any time τ when WM errs, at least half of the weight gets halved: $\Phi^{(\tau+1)} \le \frac{3}{4}\Phi^{(\tau)}$

This implies

$$\Phi^{(T+1)} \le \left(\frac{3}{4}\right)^{M_{\mathrm{WM}}} \Phi^{(1)}$$

Combining these facts yields

$$\left(\frac{1}{2}\right)^{M_i} \le \left(\frac{3}{4}\right)^{M_{\mathrm{WM}}} N$$

Taking the base-2 logarithm of both sides,

$$-M_i \le \log N + \log\left(\frac{3}{4}\right) M_{\mathrm{WM}}$$

[4] We will use log to denote the binary logarithm

so

$$M_{\text{WM}} \leq \underbrace{\frac{1}{\log(4/3)}}_{\approx 2.41}(M_i + \log N)$$

The leading constant is a consequence of our arbitrary choice to halve the weights. We can optimize ε in the update rule

$$w_i^{(t+1)} \leftarrow w_i^{(t)}/(1+\varepsilon)$$

then using the WM strategy we may achieve

$$M_{\text{WM}} \leq 2(1+\varepsilon)\big(M_i + O(\log N/\varepsilon)\big)$$

4.2 Randomized Strategies

In the exercises, you proved that there are instances for which weighted majority does twice as many mistakes as the best expert! This is undesirable. To overcome this limitation, we will allow the aggregator to play a random strategy instead of always making a deterministic prediction (that the adversary then uses to create bad instances). A side note is that this is often a general strategy: randomization is often very good to limit the effect of adversaries.
Allowing for randomized strategies leads to the following game with T days and N experts:

For $t = 1, \ldots, T$:

1. Each expert $i \in [N]$ gives some advice.
2. Allocator picks some distribution $\vec{p}^{(t)} = \left(p_1^{(t)}, \ldots, p_N^{(t)}\right)$ over the experts.
3. Adversary, with knowledge of the expert advice and $\vec{p}^{(t)}$, determines a cost vector $\vec{m}^{(t)} = \left(m_1^{(t)}, \ldots, m_N^{(t)}\right) \in [-1,1]^N$.
4. Allocator observes the cost vector and suffers $\vec{p}^{(t)} \cdot \vec{m}^{(t)} = \sum_{i\in[N]} p_i^{(t)} m_i^{(t)}$.

Note that in the above game, we have abstracted away the given advice and in fact Step 1 is not important. At each day t, the goal is to find a distribution $\vec{p}^{(t)}$ over the experts so as to minimize the suffering (the cost). For each day t, the adversary decides that the cost of following the i'th expert's advice is $m_i^{(t)}$. Here, we have generalized the cost to be anything in $[-1,1]$ instead of only counting the number of mistakes. (A negative number means that it was profitable to follow that expert's advice.)
As we play a randomized strategy, the expected cost incurred at day t is thus

$$\sum_{i\in[N]} \Pr[\text{aggregator follows expert } i \text{ at day } t] \cdot m_i^{(t)} = \sum_{i\in[N]} p_i^{(t)} m_i^{(t)} =: \vec{p}^{(t)} \cdot \vec{m}^{(t)}.$$

The ultimate goal is to design a strategy that does as well as the best expert. That is, we wish to find a strategy such that for every $i \in [N]$

$$\underbrace{\sum_{t=1}^{T} \vec{p}^{(t)} \cdot \vec{m}^{(t)}}_{\text{our loss}} \leq \underbrace{\sum_{t=1}^{T} m_i^{(t)}}_{\text{loss of best expert}} + (\text{small error terms}).$$

The *Hedge* strategy that we explain in the next section accomplishes this.

Example 4.2.1

A natural example of the above situation is when you investment money. You have the option to choose between say 4 different funds provided by Postfinance, UBS, Credit Suisse, and Banque cantonal. The

probability $\vec{p}^{(t)}$ vector that you choose at day t then corresponds to what fraction of your investment goes to the different funds. The goal is to update these fractions so as to make as much money as the best of the 4 options over the long run. Here the penalties (cost or profit) $\vec{m}^{(t)}$ that you observe reflect the performance of the different investments.

It is quite remarkable that you can do (almost) as good as the best possible option by doing rebalancing. HOWEVER, the caveat "almost" assumes that you live a very long life. For a small T, the guarantees get slightly worse.

4.3 The Hedge strategy

We now explain and analyze the Hedge strategy for the setting introduced in the last section. It is parameterized by the "learning parameter" $\varepsilon > 0$:

- Initially, assign each expert i a weight $w_i^{(1)}$ of 1. (All experts are equally trustworthy in the beginning.)

At each time t:

- Pick the distribution $p_i^{(t)} = w_i^{(t)}/\Phi^{(t)}$ where $\Phi^{(t)} = \sum_{i\in[N]} w_i^{(t)}$.
- After observing the cost vector, set $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\varepsilon \cdot m_i^{(t)}}$.

The difference between Hedge and the so-called Multiplicative Weight Update method is the weight-update $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\varepsilon \cdot m_i^{(t)}}$ instead of $w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \varepsilon \cdot m_i^{(t)})$. Both methods have similar guarantees:

Theorem 4.3.1

Suppose $\varepsilon \le 1$ and for $t \in [T]$, $\vec{p}^{(t)}$ is picked by Hedge. Then for any expert i,

$$\sum_{t=1}^{T} \vec{p}^{(t)} \cdot \vec{m}^{(t)} \;\le\; \sum_{t=1}^{T} m_i^{(t)} + \frac{\ln N}{\varepsilon} + \varepsilon T.$$

Note that in the above theorem, i can be chosen to be the best expert and thus Hedge does as well as the best expert plus some small error terms.

Proof. Similar to the analysis of the weighted majority strategy, the proof goes by analyzing the potential $\Phi^t = \sum_{i\in[N]} w_i^{(t)}$.

Lower bound on $\Phi^{(T+1)}$: We lower bound the final potential as a function of i's performance:

$$\Phi^{(T+1)} = \sum_{j\in[N]} w_j^{(T+1)} \;\ge\; w_i^{(T+1)} = \exp\Big(-\varepsilon \sum_{t=1}^{T} m_i^{(t)}\Big),$$

where the last equality follows from that the initial weight of i was one and every day t his weight was updated by $e^{-\varepsilon m_i^{(t)}}$.

Upper bound on $\Phi^{(T+1)}$: We upper bound the final potential in terms of the total cost Hedge suffered. By definition,

$$\Phi^{(t+1)} = \sum_{j\in[N]} w_j^{(t+1)} = \sum_{j\in[N]} w_j^{(t)} \cdot \exp\big(-\varepsilon m_j^{(t)}\big).$$

The exponentiated term is in $[-1,1]$. Since $e^x \le 1+x+x^2$ for $x \in [-1,1]$ (do a Taylor expansion),

$$\begin{aligned}
\sum_{j\in[N]} w_j^{(t)} \cdot \exp\big(-\varepsilon m_j^{(t)}\big) &\le \sum_{j\in[N]} w_j^{(t)}\Big(1-\varepsilon m_j^{(t)} + \varepsilon^2 \big(m_j^{(t)}\big)^2\Big) \\
&\le \sum_{j\in[N]} w_j^{(t)}\Big(1-\varepsilon m_j^{(t)} + \varepsilon^2\Big) && \text{(since } (m_j^{(t)})^2 \le 1\text{)} \\
&= \sum_{j\in[N]} w_j^{(t)}(1+\varepsilon^2) - \varepsilon \sum_{j\in[N]} w_j^{(t)} m_j^{(t)} \\
&= \Phi^{(t)}(1+\varepsilon^2) - \varepsilon \sum_{j\in[N]} \Phi^{(t)} p_j^{(t)} m_j^{(t)} && \text{(since } \Phi^{(t)} p_j^{(t)} = w_j^{(t)}\text{)} \\
&= \Phi^{(t)}\Big(1+\varepsilon^2 - \varepsilon\, \vec{p}^{(t)}\vec{m}^{(t)}\Big) \\
&\le \Phi^{(t)} \cdot \exp\Big(\varepsilon^2 - \varepsilon\, \vec{p}^{(t)}\vec{m}^{(t)}\Big) && \text{(since } 1+x \le e^x\text{).}
\end{aligned}$$

We therefore have

$$\begin{aligned}
\Phi^{(T+1)} &\le \Phi^{(T)} \cdot \exp\Big(\varepsilon^2 - \varepsilon\, \vec{p}^{(T)}\vec{m}^{(T)}\Big) \\
&\le \Phi^{(T-1)} \cdot \exp\Big(\varepsilon^2 - \varepsilon\, \vec{p}^{(T-1)}\vec{m}^{(T-1)}\Big) \cdot \exp\Big(\varepsilon^2 - \varepsilon\, \vec{p}^{(T)}\vec{m}^{(T)}\Big) \\
&\ \ \vdots \\
&\le \Phi^{(1)} \cdot \exp\Big(\varepsilon^2 T - \varepsilon \sum_{t=1}^{T} \vec{p}^{(t)}\vec{m}^{(t)}\Big) \\
&= N \cdot \exp\Big(\varepsilon^2 T - \varepsilon \sum_{t=1}^{T} \vec{p}^{(t)}\vec{m}^{(t)}\Big),
\end{aligned}$$

where for the equality we used that $\Phi^{(1)} = N$ since each expert was initialized with a weight of 1.
The above bounds give us

$$\exp\Big(-\varepsilon \sum_{t=1}^{T} m_i^{(t)}\Big) \ \le\ \Phi^{(T+1)} \ \le\ N \cdot \exp\Big(\varepsilon^2 T - \varepsilon \sum_{t=1}^{T} \vec{p}^{(t)}\vec{m}^{(t)}\Big).$$

Taking (natural) logarithms,

$$-\varepsilon \sum_{t=1}^{T} m_i^{(t)} \ \le\ \ln(N) + \varepsilon^2 T - \varepsilon \sum_{t=1}^{T} \vec{p}^{(t)}\vec{m}^{(t)}$$

and the final result follows by dividing by ε and rearranging the terms. ☺

Remark The above proof may seem messy with all the inequalities. However, it follows a standard "framework": upper and lower bound the potential function. These Multiplicative Weight Update type of algorithms are most often analyzed using this potential argument. Once you know that cool argument, the analysis is almost automatic modulo some rewriting.
For the application of solving covering LPs, it will be convenient to consider the average cost incurred per day. We also generalize so that the cost vector can take values in $[-\rho,\rho]^N$ where ρ is called the "width". The proof is the same by scaling and we get the following corollary:

Corollary 4.3.1
Suppose $\varepsilon \le 1$ and for $t \in [T]$, $\vec{p}^{(t)}$ is picked by Hedge and cost vectors satisfy $\vec{m}^{(t)} \in [-\rho,\rho]^N$. If

$T \geq (4\rho^2 \ln N)/\varepsilon^2$, then for any expert i:

$$\frac{1}{T}\sum_{t=1}^{T} \vec{p}^{(t)} \cdot \vec{m}^{(t)} \;\leq\; \frac{1}{T}\sum_{t=1}^{T} m_i^{(t)} + 2\varepsilon.$$

4.4 Hedging for LPs

We now turn to a nice application of the Hedge method to that of solving covering LPs. We start by defining these LPs.

4.4.1 Covering LPs

Definition 4.4.1: Covering Linear Program

A linear program of the form:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & Ax \geq b \\ & 1 \geq x_j \geq 0 \quad \forall j \end{aligned}$$

is a *covering linear program* if

$$A \in \mathbb{R}_+^{m \times n}, \quad b \in \mathbb{R}_+^m \quad \text{and} \quad c \in \mathbb{R}_+^n$$

This definition simply ensures that all the coefficients of the constraints and of the objective function are non-negative. Notice that both the set cover relaxation and the vertex cover relaxation that we saw in class were covering LPs.
Let us now introduce an example of a covering LP that we will reuse later:

Example 4.4.1 (Covering LP)

$$\begin{aligned} \text{minimize} \quad & x_1 + 2x_2 \\ \text{subject to} \quad & x_1 + 3x_2 \geq 2 \\ & 2x_1 + x_2 \geq 1 \\ & 1 \geq x_1, x_2 \geq 0 \end{aligned} \tag{1}$$

4.4.2 General idea

The idea of using the Hedge method for linear programming is to associate an expert with each constraint of the LP. In other words, the Hedge method will maintain a weight distribution over the set of constraints of a linear problem to solve, and to iteratively update those weights in a multiplicative manner based on the cost function at each step/day. Of course we need to define the cost function in a smart way later but let us first define the notion of an oracle and then give a small instructive example.
Initially, the Hedge method will give a weight $w_i^{(1)} = 1$ for every constraint/expert $i = 1, \ldots, m$ (the number m of constraints now equals the number of experts). And at each day t, it will maintain a convex combination $\vec{p}^{(t)}$ of the constraints (that is defined in terms of the weights). Using such a convex combination $\vec{p}$, a natural easier LP with a single constraint is obtained by summing up all the constraints according to $\vec{p}$. Any optimal solution of the original LP is also a solution of this reduced problem, so the new problem will have at most the same cost as the previous one. We define an oracle for solving this reduced problem:

Definition 4.4.2: Oracle for Reduced Linear Problem

An oracle that, given $\vec{p} = (p_1, \ldots, p_m) \geq 0$ such that $\sum_{i=1}^{m} p_i = 1$, outputs an optimal solution x^* to the following reduced linear problem:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \Big(\sum_{i=1}^{m} p_i A_i\Big) \cdot x \;\geq\; \sum_{i=1}^{m} p_i b_i \\ & 1 \geq x \geq 0 \end{aligned}$$

This linear problem is in practice much easier to solve than the original one and we can design a simple greedy algorithm for the oracle.
For concreteness, let us consider a small example.

Example 4.4.2 (Reduced LP)

If we apply the method we described to our example 4.4.1, we have two initials weights $w_1 = w_2 = 1$ and thus $p_1 = p_2 = \frac{1}{2}$, and we sum all the constraints:

$$\begin{aligned} & p_1(x_1 + 3x_2) + p_2(2x_1 + x_2) \geq p_1 \cdot 2 + p_2 \cdot 1 \;\Leftrightarrow \\ & 1.5x_1 + 2x_2 \geq 1.5 \\ & 1 \geq x_1, x_2 \geq 0 \end{aligned}$$

By using the oracle, an optimal solution to this reduced problem is $x_1 = 1$, $x_2 = 0$ of cost 1. But is this a feasible solution to our original problem? By checking the constraints of the original LP:

$2x_1 + x_2 = 2 \geq 1$	OK
$x_1 + 3x_2 = 1 < 2$	not OK

We will need to go back to the original problem and increase the weights of the unsatisfied constraints to give them more importance and decrease the weights of the satisfied constraint. We will perform these updates (as done by Hedge) multiplicatively that we describe in detail later. We first describe how to implement the oracle in general.

4.4.3 Implementation of the oracle

The oracle is given an objective function minimize $\sum_{i=1}^{n} c_i x_i$ and only one constraint that we can rewrite as $\sum_{i=1}^{n} d_i x_i \geq b$ (which is the weighted sum of all constraints). We also have $1 \geq x_i \geq 0 \ \forall i$ and $c_i, d_i \geq 0 \ \forall i$ since it is a covering problem.
The idea is to assign the maximum value (namely 1) to the variables that have the highest ratio constraint coefficient/objective coefficient. That way we will satisfy the constraint as fast as possible while maintaining a small objective function. Then assign zero to the other variables and possibly an intermediate value for the variable that is at the limit to make the constraint satisfied. This is very similar to the most "bang-for-the-buck" greedy and it is the solution to fractional knapsack. Formally:

- Sort and relabel all variables x_i so that $d_1/c_1 \geq d_2/c_2 \geq \ldots \geq d_n/c_n$.
- Let $k = \min\{j : \sum_{i=1}^{j} d_i \geq b\}$ and $\ell = b - \sum_{i=1}^{k-1} d_i$.
- Set $x_i := 1$ for $i = 1, \ldots, k-1$.
- Set $x_k := \ell / d_k$.
- Set $x_i := 0$ for $i = k+1, \ldots, n$.

We have,

$$\sum_{i=1}^{n} d_i x_i = \sum_{i=1}^{k-1} d_i x_i + d_k x_k + \sum_{i=k+1}^{n} d_i x_i = \sum_{i=1}^{k-1} d_i + \ell = b$$

Therefore the constraint is exactly satisfied. By the ordering of the variables, this ensures that we minimize the objective function as required. Having implemented the oracle, we proceed to formally define the Hedge algorithm for linear programming.

4.4.4 Hedge algorithm for covering LPs

As already explained, we associate an expert to each constraint of the covering LP. In addition, as hinted above, we wish to increase the weight of unsatisfied constraints and decrease the weight of satisfied constraints (in a smooth manner depending on the size of the violation or the slack). The Hedge algorithm for covering LPs thus becomes:

- Assign each constraint i a weight $w_i^{(1)}$ initialized to 1.

At each time t:

- Pick the distribution $p_i^{(t)} = w_i^{(t)}/\Phi^{(t)}$ where $\Phi^{(t)} = \sum_{i\in[N]} w_i^{(t)}$.
- Now we define the cost vector instead of the adversary as follows:
  - Let $x^{(t)}$ be the solution returned by the oracle on the LP obtained by using the convex combination $\vec{p}^{(t)}$ of constraints. Notice that cost of $x^{(t)}$, i.e., $c^\top x^{(t)}$, is at most the cost of an optimal solution to the original LP.
  - Define the cost of constraint i as

$$m_i^{(t)} = \sum_{j=1}^{n} A_{ij} x_j - b_i = A_i x - b_i.$$

  Notice that we have a positive cost if the constraint is satisfied (so the weight will be decreased by Hedge) and a negative cost if it is violated (so the weight will be increased by Hedge).

- After observing the cost vector, set $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\varepsilon \cdot m_i^{(t)}}$.

Output: the average $\bar{x} = \frac{1}{T}\sum_{t=1}^{T} x^{(t)}$ of the constructed solutions.

Analysis. Since the analysis for the Hedge algorithm works with an adversarial construction of the cost vectors, it definitely holds for the cost vectors we constructed. Let

$$\rho = \max_{1\le i\le m} \{\max(b_i,\, A_i\mathbf{1} - b_i)\},$$

be (an upper bound on) the width of our constructed cost vectors. By Corollary 3, we thus have for $\varepsilon \in [0,1]$, $T \ge (4\rho^2 \ln m)/\varepsilon^2$ and any constraint i that

$$\frac{1}{T}\sum_{t=1}^{T} \vec{p}^{(t)} \cdot \vec{m}^{(t)} \;\le\; \frac{1}{T}\sum_{t=1}^{T} m_i^{(t)} + 2\varepsilon. \tag{2}$$

Let us first consider the left-hand-side of the above expression. We have that

$$\sum_{t=1}^{T} \vec{p}^{(t)} \cdot \vec{m}^{(t)} = \sum_{t=1}^{T} \Big(\sum_i p_i^{(t)} \cdot m_i^{(t)} \Big) = \sum_{t=1}^{T} \underbrace{\Big(\sum_i p_i^{(t)} \cdot \big(A_i x^{(t)} - b_i\big) \Big)}_{(*)}$$

which is non-negative because $(*)$ is non-negative for every t since the oracle outputs a feasible solution. Using that the left-hand-side of (2) is non-negative, we get

$$-2\varepsilon \;\leq\; \frac{1}{T}\sum_{t=1}^{T} m_i^{(t)} = \frac{1}{T}\sum_{t=1}^{T}\left(A_i x^{(t)} - b_i\right) = A_i\bar{x} - b_i$$

which implies that

$$A_i\bar{x} \;\geq\; b_i - 2\varepsilon,$$

for every constraint i. So our solution $\bar{x}$ is almost feasible. Moreover the cost $c^\top\bar{x} = c^\top\left(\frac{1}{T}\sum_{t\in[T]} x^{(t)}\right)$ is at most the cost of an optimal solution to the original LP since each $x^{(t)}$ has no higher cost than an optimal solution (by the properties of the oracle).

Setting the parameters for solving the set cover relaxation. For set cover we have that $\rho \leq n$ and therefore, it is sufficient to set $T = (4n^2 \ln m)/\varepsilon^2$ (this can in fact be improved by a better analysis to $\approx n \ln m/\varepsilon^2$). This gives a solution $\bar{x}$ that satisfies $\sum_{e\in S} x_e \geq 1 - 2\varepsilon$ for every set S and the cost of $\bar{x}$ is at most that of an optimal LP solution. We can obtain a feasible (approximately optimal) solution by simply defining $x^* = \frac{1}{1-2\varepsilon}\bar{x}$.